Extending Data Dependencies to Unstructured Data

Yuanzhe Chen[1], Wenfei Fan[1,2,3], Kaiwen Lin[1], Ping Lu[3], Yaoshu Wang[1], Min Xie[1]
[1]Shenzhen Institute of Computing Sciences [2]University of Edinburgh [3]Beihang University
{chenyuanzhe, linkaiwen, yaoshuw, xiemin}@sics.ac.cn, wenfei@inf.ed.ac.uk, luping@buaa.edu.cn

***Abstract*—This paper extends data dependencies beyond relational data to unstructured data via *Data Cleaning Rules* (DCRs). Unlike traditional dependencies, DCRs combine logical predicates on relational data with ML models over unstructured data, supporting multi-modal error detection. We show that satisfiability, implication and validation for DCRs have the same complexity as their relational counterparts. We further develop a distillation-based model for cross-modal entity alignment, and a learning-based algorithm for discovering DCRs. Using real-life data, we experimentally verify that DCRs effectively detect errors across modalities while remaining computational efficient.**

I. Introduction

Data dependencies are central to database systems, providing powerful tools to express data consistency and integrity. Since Codd introduced functional dependencies (FDs) [1], a variety of extensions, *e.g.,* denial constraints (DCs) [2], conditional functional dependencies (CFDs) [3] and entity-enhancing rules (REEs) [4], have been developed to support error detection and data cleaning in relational data.

Modern data, however, is increasingly multi-modal. In applications such as electronic health records (EHRs), e-commerce and social platforms, relational data is tightly coupled with unstructured data such as text, images and audio [5]. For example, EHRs in the United States were increasingly adopted from 9.4% to 83.8% within seven years [6], which combine tableau data (*e.g.,* patient demographics) and unstructured data (*e.g.,* clinical notes and medical images). Joint analytics of structured and unstructured data substantially improves predictive accuracy of machine learning (ML) models [7].

While such data enriches downstream analytics, it also introduces a fundamental challenge: *ensuring consistency across modalities*. For example, empirical studies show that at least half of EHRs contain inaccuracies [8], from data entry errors, copy-and-paste propagation of outdated records, to incorrect clinical observations [9]. These issues are further exacerbated by limited interoperability across healthcare systems, which leads to missing and incomplete information [10].

Existing dependencies are *incapable of* capturing cross-modal consistency. They are designed for fixed-schema relational data and operate only over tuples, lacking the ability to (a) extract information from unstructured data, (b) align entities across heterogeneous modalities, or (c) reason jointly over structured and unstructured representations. As a result, many real-world inconsistencies remain undetectable.

Example 1: Consider a *real* case on a healthcare dataset [11], comprising tabular features and skin-lesion images. The logic rule φ_0 below can be used to check diagnostic consistency.

φ_0 : A patient is inferred to have BCC (basal cell carcinoma) if (a) his symptoms (*e.g.,* bleeding, color pattern) and image-derived features (*e.g.,* surface) meet the diagnostic criteria, and (b) his skin-lesion image is similar to a confirmed BCC case.

To enable such checking, we need to (a) link unstructured data (*e.g.,* images) to tuples, (b) extract relevant features from both structured and unstructured sources, and (c) reason across modalities. These are beyond classical dependencies. □

Problem. The practical need raises a key question: *Can we extend data dependencies to mixed datasets of relational and unstructured data, enabling systematic cross-modal consistency checking without converting unstructured data to relations?*

Challenges. This extension is nontrivial due to:

- *Lack of schema.* Unstructured data lacks fixed attributes, making predicate definition difficult.
- *Cross-modal alignment.* Linking tuples with unstructured objects is ambiguous and requires learned representations.
- *Joint reasoning.* Logical reasoning must integrate ML signals while preserving formal semantics and efficiency.

Our approach. We observe that dependencies over mixed data can be achieved by extending the types of predicates they support, to include three capabilities: (a) extracting structured information from unstructured data, (b) aligning entities across different modalities, and (c) reasoning jointly over structured and unstructured representations. This leads to a unified framework that combines logical constraints with ML signals.

Based on these, we introduce *Data Cleaning Rules* (DCRs), a new class of dependencies defined over mixed datasets. DCRs extend relational rules with (a) ML-based extraction functions, (b) a heterogeneous entity resolution (HER) operator, and (c) predicates that operate across modalities. Despite this increased expressiveness, we show that reasoning about DCRs retains the same complexity as classical dependencies.

Contributions & Organization. This paper provides a theoretical foundation along with practical models and algorithms for multi-modal consistency checking with DCRs.

(1) Dependency model (Section II). We introduce DCRs, extending relational dependencies (*e.g.,* REEs [12], [13]) to support extraction, alignment, and cross-modal reasoning.

(2) Theory (Section III). We show that DCRs preserve the complexity of REEs: satisfiability, implication and validation remain NP-complete, Π_2^p-complete and coNP-complete, respectively. This establishes theoretical feasibility of adapting prior reasoning techniques to DCRs without adding overhead.

(3) Model for HER (Section IV). We train a fusion-based model for HER via distillation without human annotations, to align tuples and unstructured objects to identify entities.

(4) Algorithm for DCR Learning (Section V). We cast DCR discovery as a Markov Decision Process (MDP) and solve it using Monte Carlo Tree Search (MCTS), guided by learned models for efficient exploration of the multi-modal rule space.

(5) Experimental study (Section VI). Using real-life datasets, we empirically find the following. (a) DCRs effectively detect inconsistencies across relational and unstructured data. Its average F1 score is 0.83, outperforming baselines by 0.62. (b) Incorporating unstructured data boosts the F1 score from 0.38 to 0.80 on Fakeddit [14], showing the value of multi-modal error detection. (c) Our learning-based DCR discovery algorithm is efficient, taking 54.9min on a dataset with 5.01M instances. (d) The accuracy of our method for linking relational tuples to unstructured objects reaches 0.84 on average.

We discuss related work in Section VII and summarize the novelty in Section VIII. Detailed proofs are deferred to [15].

Impact. This work extends dependencies beyond relations, providing a foundation for multi-modal consistency checking and bridging database theory and AI-driven processing.

II. Data Cleaning Rules

This section introduces Data Cleaning Rules (DCRs). We present mixed datasets and predicates over the data in Section II-A, and define DCRs with the predicates in Section II-B.

A. Predicates over Mixed Datasets

We start with datasets, structured, unstructured and mixed.

Relational data. A database schema is $\mathcal{R} = (R_1, \ldots, R_m)$, where each R_j is a relation schema $R(A_1, \ldots, A_n)$ with attributes A_i ($i \in [1, n]$), where each A_i can be a numerical attribute (*e.g.,* weight), a categorical attribute (*e.g.,* gender) or a textual attribute (*e.g.,* name). A dataset $\mathcal{D}$ of $\mathcal{R}$ is $(D_1, \ldots, D_m)$, where each D_i is a relation of R_i.

Unstructured data. We consider a collection $\mathcal{U}$ of unstructured data that does not adhere to a predefined schema, *i.e.,* it lacks fixed attributes. This includes documents, images, audio and video. For documents, each $u \in \mathcal{U}$ may be a paragraph, or sentence, with tokens as the smallest units; for video/audio, the smallest units are frames, short clips or segments.

Mixed datasets. A mixed dataset is $(\mathcal{D}, \mathcal{U})$, where $\mathcal{D}$ is a relational dataset of schema $\mathcal{R}$, and $\mathcal{U}$ is a collection of unstructured data. We use t, s for tuples in $\mathcal{D}$, o for objects extracted from $\mathcal{U}$, and e for either a tuple or an object.

Predicates. *Predicates* over a mixed dataset $(\mathcal{D}, \mathcal{U})$ are:

$$p \quad ::= \quad R(t) \;\mid\; o = \mathcal{M}_U(\mathcal{U}, t) \;\mid\; \Theta_{\asymp}(t, o) \;\mid\; \mathcal{M}_R(e_1, e_2) \;\mid\; e.A \oplus c \;\mid\; e_1.A \oplus e_1.B,$$

where $\oplus \in \{=, \neq, <, \leq, >, \geq\}$. Predicates are classified as binding, HER, ML, and logic predicates, as follows.

(1) Binding predicates. (a) As in tuple relational calculus [16], $R(t)$ is a relation atom that binds *tuple variable* t ($R \in \mathcal{R}$). (b) $\mathcal{M}_U$ is an existing ML model over unstructured data $\mathcal{U}$ and an (optional) tuple t; it extracts unstructured objects relevant to t, where an object is an image or video frame, audio segment, a classification label [17], an entity [18], key-value pairs [19], or text via summarization/extraction/conversion [20], [21], [22]. (c) In $o = \mathcal{M}_U(\mathcal{U}, t)$, o is an *object variable* bound by $\mathcal{M}_U(\mathcal{U}, t)$. If multiple objects are returned, o denotes one of them.

To exemplify, (a) for documents, we can use Bert-MRC [18] as $\mathcal{M}_U$ to return entities, by formulating a NER (Named Entity Resolution) task, and extract key-value pairs or labels based on the well-known Bert [19]; (b) for image/video, we can use Qwen2.5-VL [20], [21] or OCR (Optical Character Recognition) models such as Tesseract [22] as $\mathcal{M}_U$ to return captions or text characters from image/video; and (c) for audio, we use $\mathcal{M}_U$ to return labels by first adopting BEATS [17] to tokenize audio inputs, and then training a model to predict the label.

(2) HER operator. For heterogeneous entity resolution (HER), $\Theta_{\asymp}(t, o)$ tests whether a tuple t and an object o (extracted by $\mathcal{M}_U(\mathcal{U}, t)$) refer to the same real-world entity (see details in Section IV). It returns true if so, and false otherwise.

(3) ML predicates. An ML predicate $\mathcal{M}_R$ is a pre-trained model that returns a Boolean, *e.g.,* $\mathcal{M}_{\mathsf{reg}} \geq \delta$ for a regression model with threshold δ, or a VLM over multi-modal inputs.

Model $\mathcal{M}_R$ applies to (e_1, e_2) where e_1, e_2 are tuples or objects. When both are tuples, $\mathcal{M}_R$ instantiates standard models on relational data (*e.g.,* NLP models [19], entity resolution (ER), or error detection/correction [23]). More generally, $\mathcal{M}_R$ supports cross-modal inputs, where e_1 is a tuple (or its serialized text) and e_2 is a tuple or an unstructured object (*e.g.,* image or video frame). Modern VLMs (*e.g.,* BLIP-2 [24], Video-LLaVA [25], Qwen3-VL [26], Qwen2.5-VL [20]) align multi-modal inputs and return Boolean similarity decisions.

(4) Logic predicates. For a tuple or an extracted object e with attribute A, $e.A$ denotes its value. We call $e.A \oplus c$ a *constant predicate* and $e.A \oplus e.B$ a *variable predicate*, which capture duplicates, semantic conflicts or timeliness (see Section II-B).

B. Data Cleaning Rules over Mixed Datasets

A *data cleaning rule* (DCR) over mixed datasets $(\mathcal{D}, \mathcal{U})$ is

$$\varphi = X \rightarrow p_0,$$

where the precondition X is a conjunction $\bigwedge p$ of predicates over $(\mathcal{D}, \mathcal{U})$, and the consequence p_0 is a predicate over $(\mathcal{D}, \mathcal{U})$ whose tuple and object variables are all bound in X.

Example 2: A DCR for diagnostic consistency (Example 1) is:

(1) $\varphi_0 : \mathsf{patient}(t) \wedge \mathsf{patient}(t') \wedge o = \mathcal{M}_U(\mathsf{image}, t) \wedge o' = \mathcal{M}_U(\mathsf{image}, t') \wedge \Theta_{\asymp}(t, o) \wedge \Theta_{\asymp}(t', o') \wedge t.\mathsf{bleed} = \mathrm{True} \wedge t.\mathsf{colorPattern} = \mathrm{variegated} \wedge o.\mathsf{surfaceType} = \mathrm{smooth} \wedge t'.\mathsf{diagnostic} = \mathrm{BCC} \wedge \mathcal{M}_{\mathsf{sim}}(o, o') \rightarrow t.\mathsf{diagnostic} = \mathrm{BCC}$, where $\mathcal{M}_U$ is a VLM with handcrafted prompts for lesion surface extraction, and $\mathcal{M}_{\mathsf{sim}}$ is an image embedding model for semantic similarity. This DCR jointly reasons over structured and image data, reinforced by cross-patient image similarity.

Below are more DCRs for detecting *real EHR errors* reported in [27]. We consider the following mixed dataset:

- *Relations.* Tables with schemas patient(pid, name, gender, weight, GFR (glomerular filtration rate), allergy, ...), and prescription(id, pid, doctor, medication, lesion, ...).
- *Documents,* such as clinical notes and doctors' instructions.
- *Other modalities,* including medical images (*e.g.,* X-rays, CT) and voice recordings for consultation or notification.

(2) φ_1 : patient$(t) \wedge$ patient$(t') \wedge$ prescription$(s) \wedge t$.pid $= s$.pid $\wedge \mathcal{M}_{\mathsf{ER}}(t, t') \wedge s$.medication $=$ Amoxicillin $\wedge\, t'$.allergy $=$ Penicillin $\rightarrow t$.pid $\neq t'$.pid, where $\mathcal{M}_{\mathsf{ER}}$ is a model for checking whether two patients are the same person. It overrides $\mathcal{M}_{\mathsf{ER}}$ by domain knowledge: although $\mathcal{M}_{\mathsf{ER}}$ predicates a match, t and t' differ as amoxicillin is a penicillin antibiotic and patients allergic to penicillin cannot be prescribed it.

(3) A patient-reported error about demographics is "reference to my left testicle (I'm female)" [27]. Given a set $\mathcal{U}$ of doctor's instructions, we can detect such error using the DCR:

φ_2 : patient$(t) \wedge o = \mathcal{M}_U$(instructions$, t) \wedge \Theta_{\asymp}(t, o) \wedge o$.treatment_area $=$ testicle $\rightarrow t$.gender $=$ Male. It states that if the treatment area is "testicle" in the instructions, the corresponding patient, matched via $\Theta_{\asymp}(\cdot)$, must be a male.

(4) Another incorrect medication reported is "an incorrect dosage that was being used by a referred physician for an infusion" [27]. Consider a set $\mathcal{U}$ of clinical notes, we have:

φ_3 : patient$(t) \wedge o = \mathcal{M}_U$(notes$, t) \wedge \Theta_{\asymp}(t, o) \wedge t$.weight $\leq$ 70kg $\wedge\, t$.GFR $\leq$ 60 mL/min $\wedge\, o$.physician_type = referred $\wedge o$.medication $=$ Amoxicillin $\rightarrow o$.dosage $\leq$ 700mg. This rule suggests the maximum dose of Amoxicillin during treatment, since the reduced kidney function can impair the elimination of the drug from the body, increasing the risk of toxicity.

(5) Cross-source inconsistencies may arise, *e.g.,* a patient is informed of two positive lymph nodes while imaging reports three [27]. Such cases are detected by the following DCR:

φ_4 : patient$(t) \wedge$ prescription$(s) \wedge o = \mathcal{M}_U$(image, t) $\wedge$ $\Theta_{\asymp}(t, o) \wedge t$.pid $= s$.pid $\wedge\, \mathcal{M}_{\mathsf{ts}}(s, o) \wedge t$.lesion $=$ lymph $\wedge$ o.lesion $=$ lymph $\rightarrow t$.lesion_num $= o$.lesion_num, where $\mathcal{M}_{\mathsf{ts}}$ verifies that the prescription record s and the image object o refer to the same prescription episode by their timestamps. This rule enforces cross-source consistency: for a given patient and prescription, the reported number of lymph nodes must agree between structured records and image-derived evidence.

(6) Audio calls between a healthcare center and a patient can also be used for error detection, as elaborated by the DCR:

φ_5 : patient$(t) \wedge$ prescription$(s) \wedge o = \mathcal{M}_U$(call$, t) \wedge t$.pid $=$ s.pid $\wedge\, s$.test $=$ NIPT (non-invasive prenatal testing) $\wedge o$.label $=$ warning $\rightarrow s$.risk_level $=$ high, where $\mathcal{M}_U$ identifies the purpose of a call (*e.g.,* to notify a risky test result). It states that if the healthcare center calls a patient for warning the test result of NIPT, its risk level must be classified as high. □

Semantics. A *valuation* h of a DCR φ maps tuple variables to tuples in $\mathcal{D}$ and object variables to objects extracted by $\mathcal{M}_U$.

We say that a valuation h satisfies a predicate p, written $h \models p$, as follows. (1) If p is $R(t)$, then $h \models p$ iff $h(t)$ is a tuple of R in $\mathcal{D}$. If p is $o = \mathcal{M}_U(\mathcal{U}, t)$, then $h \models p$ iff $h(o)$ is returned by $\mathcal{M}_U(\mathcal{U}, h(t))$. (2) If p is $\Theta_{\asymp}(t, o)$, then $h \models p$ iff $h(t)$ and $h(o)$ refer to the same entity. (3) If p is $\mathcal{M}_R(e_1, e_2)$, then $h \models p$ iff $\mathcal{M}_R(h(e_1), h(e_2))$ is true. (4) If p is $e.A \oplus c$, then $h \models p$ iff $h(e).A \oplus c$; similarly for $e_1.A \oplus e_2.B$.

For a precondition X, we write $h \models X$ if $h \models p$ for all $p \in X$. For a DCR $\varphi = X \rightarrow p_0$, we write $h \models \varphi$ if $h \models X$ implies $h \models p_0$. A mixed dataset $(\mathcal{D}, \mathcal{U})$ *satisfies* φ, denoted $(\mathcal{D}, \mathcal{U}) \models \varphi$, if $h \models \varphi$ for all valuations h. For a set Σ of DCRs, $(\mathcal{D}, \mathcal{U}) \models \Sigma$ if $(\mathcal{D}, \mathcal{U}) \models \varphi$ for all $\varphi \in \Sigma$.

The notations of the paper are summarized in Table I.

Notations	Definitions
$(\mathcal{D}, \mathcal{U})$	a mixed dataset
t, o	a tuple in $\mathcal{D}$ and an object from $\mathcal{U}$
$\varphi : X \rightarrow p_0$	a data cleaning rule (DCR)
req$(\mathcal{P}_X, \mathcal{P}_0, \sigma, \delta, \eta)$	the discovery requirement (*e.g.,* supp, conf)
$\mathcal{M}_U$, $\Theta_{\asymp}(t, o)$	the external ML model and HER operator
$\mathcal{M}_R$	ML predicates for *e.g.,* semantic matching
$\mathcal{S}, \mathcal{A}, \mathbb{R}$	state space, action space and reward function
$N(s)$, $Q(s)$	the visit count and expected reward of s
$\mathcal{M}_p$, $\mathcal{M}_v$	the policy model and value model
$\hat{Q}(s)$, $Q_v(s)$	an empirical value and an ML-estimated value

TABLE I: Notations

Remark. DCRs subsume classical dependencies REEs [12], DCs [2], CFDs [3] and MDs [28] as special cases. Indeed, DCRs extend REEs with predicates $o = \mathcal{M}_U(\mathcal{U}, t)$, $\Theta_{\asymp}(t, o)$, $\mathcal{M}_R(e_1, e_2)$, $e.A \oplus c$ and $e_1.A \oplus e_2.B$ on unstructured data and across relational and unstructured data. DCs, CFDs and MDs are expressible as REEs (see conversion in [12], [29]).

Note that DCRs are *not* a straightforward extension of classical dependencies. Classical dependencies operate exclusively over structured, single-relation data and cannot express relationships involving unstructured objects or heterogeneous entity alignment. In contrast, DCRs introduce *new semantic constructs* that connect relational tuples with objects extracted from text, images, audio and video through ML predicates and a dedicated heterogeneous entity-resolution operator. These mechanisms enable DCRs to detect cross-modal errors that classical dependencies are fundamentally unable to capture.

Errors. A *violation* of $\varphi : X \rightarrow p_0$ is a valuation h that $h \models X$ but $h \not\models p_0$. It is an *error* detected by φ as h witnesses $(\mathcal{D}, \mathcal{U}) \not\models \varphi$. Here multiple errors detected on the same tuple are simply listed as separate violations; they do not interfere with one another as long as the rules are themselves consistent (Section III). In practice, errors must be reviewed manually or validated against master data before any correction is applied.

One can verify that all errors identified by DCRs above (except φ_1 which can be expressed as an REE [12]) cannot be caught by existing data quality rules since they do not support (1) $\mathcal{M}_U$ that extract objects from unstructured data $\mathcal{U}$, and (2) HER $\Theta_{\asymp}(t, o)$ that links a tuple t of $\mathcal{D}$ and an object o from $\mathcal{U}$.

III. Reasoning about DCRs

This section analyzes the reasoning complexity, by studying three fundamental problems for DCRs, namely, satisfiability, implication and validation, to assess the expressive power.

Satisfiability. We start with the *satisfiability* problem.

- Input: A set Σ of DCRs.
- Question: Does there exist a nonempty mixed dataset $(\mathcal{D},\mathcal{U})$ such that $(\mathcal{D},\mathcal{U}) \models \Sigma$?

Implication. The *implication problem* is stated as follows.

- Input: A set Σ of DCRs and another DCR φ.
- Question: Does Σ *implies* φ, denoted by $\Sigma \models \varphi$? Here $\Sigma \models \varphi$ if for all $(\mathcal{D},\mathcal{U})$, if $(\mathcal{D},\mathcal{U}) \models \Sigma$ then $(\mathcal{D},\mathcal{U}) \models \varphi$.

Validation. We also study the complexity of error detection.

- Input: A mixed dataset $(\mathcal{D},\mathcal{U})$ and a set Σ of DCRs.
- Question: Does $(\mathcal{D},\mathcal{U}) \models \Sigma$?

Each problem plays a distinct role in the lifecycle of dependency management: *satisfiability* ensures that rules are logically consistent and do not impose impossible requirements; *implication* identifies redundant rules, which is essential for reducing validation cost and maintaining a minimal rule set; and *validation* checks whether data satisfies the rules, forming the basis of profiling and error detection. Together, these tasks support the end-to-end process of designing, optimizing, and enforcing dependencies; in particular, implication analysis is a necessary component of rule discovery (Section V).

The satisfiability, implication and validation problems for REEs are NP-complete, Π_2^p-complete and coNP-complete, respectively [12]. Here Π_2^p denotes the class of problems that can be solved in coNP using an NP oracle (*i.e.,* $\Pi_2^p = \text{coNP}^{\text{NP}}$).

We show that reasoning about DCRs across relational and unstructured data does not make our lives harder (see [15]).

Theorem 1: *The satisfiability, implication and validation problems for* DCRs *are* NP*-complete,* Π_2^p*-complete and* coNP*-complete, respectively.* □

Proof sketch: The lower bounds of the implication and validation problems follow from their counterparts for REEs [12]. For satisfiability, however, a new proof is required: a direct reduction from REEs does not apply, as DCRs are defined over mixed datasets. Indeed, if a set Σ_R of REEs is unsatisfiable over any relational dataset, then any mixed dataset $(\emptyset,\mathcal{U})$ with $\mathcal{U} \neq \emptyset$ trivially satisfies Σ_R, when viewed as DCRs.

Satisfiability. For the upper bound, we construct a finite set of canonical mixed datasets, and reduce a set Σ of DCRs to Horn formulas of polynomial size. Then Σ is satisfiable *iff* (if and only if) at least one resulting Horn set is satisfiable, from which a mixed dataset $(\mathcal{D},\mathcal{U})$ satisfying Σ can be constructed. This yields an NP algorithm for checking satisfiability.

We prove NP-hardness by a reduction from the NP-complete 3-colorability problem [30]. Given a graph G, we construct a set of DCRs $\Sigma_D \cup \Sigma_U$ over a mixed dataset such that G admits a valid 3-coloring iff $\Sigma_D \cup \Sigma_U$ is satisfiable.

Implication. Given a set Σ of DCRs and a DCR φ, we reduce implication checking to the satisfiability of a set Σ_H^φ of Horn formulas constructed from Σ and φ. While Σ_H^φ may contain exponentially many formulas, it has only polynomially many variables, and $\Sigma \not\models \varphi$ if and only if Σ_H^φ is satisfiable and yields a mixed dataset that satisfies Σ but violates φ. This leads to a Π_2^p decision procedure for implication.

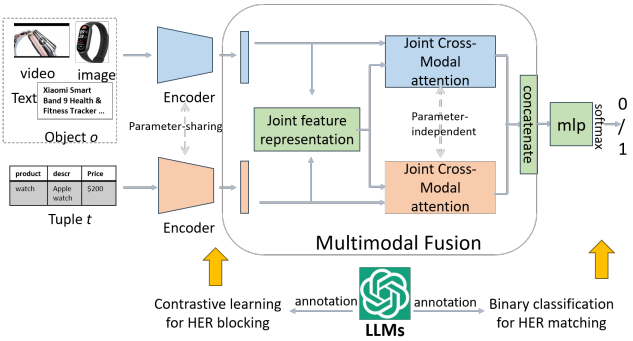


Fig. 1: The Network Architecture of HER

Validation. We decide whether $(\mathcal{D},\mathcal{R}) \not\models \Sigma$ by guessing a rule $\varphi \in \Sigma$ and a valuation h, and verifying that h violates φ. The algorithm is correct by definition and runs in coNP, since the verification step is in PTIME assuming that pretrained ML models in Σ can be evaluated in polynomial time [31], [32]. □

IV. Heterogeneous Entity Resolution

This section presents our designated HER operator $\Theta_{\asymp}$.

Heterogeneous entity resolution (HER). HER is to identify entities across modalities. Given a relational tuple t from $\mathcal{D}$ and an unstructured object o extracted from $\mathcal{U}$, $\Theta_{\asymp}(t,o)$ returns true if t and o refer to the same real-world entity. If yes, (t,o) is referred to as a *match*. Otherwise, (t,o) is a *mismatch*.

Distillation-based HER. One may want to implement the HER operation by simply matching the unique IDs carried on both structured and unstructured data, such as patient IDs for EHRs, SKUs for products, and tracking numbers for orders. However, such IDs are often unavailable for analysis in practice. Thus, we propose a more robust distillation-based HER method for matching tuples with unstructured objects.

We train a multi-modal ML model $\mathcal{M}_{\text{HER}}$ to implement $\Theta_{\asymp}(t,o)$, *i.e.,* $\mathcal{M}_{\text{HER}}(t,o)$ outputs true iff (t,o) makes a match. Note that most of existing feature alignment and fusion methods [33] cannot be directly used since HER must be both effective and human-efficient to scale to large-scale datasets.

Overview. As shown in Figure 1, $\mathcal{M}_{\text{HER}}$ consists of three parts: (1) a multi-modal representation encoder $\mathcal{M}_{\text{rep}}$ that encodes t and o into feature representations $\mathbf{t}$ and $\mathbf{o}$, respectively; (2) a multi-modal fusion layer $\mathcal{M}_{\text{fusion}}$ that integrates and aligns $\mathbf{t}$ and $\mathbf{o}$ into a unified representation $\mathbf{X}_{t,o}$; and (3) a binary classifier $\mathcal{M}_{\text{bin}}$ that processes $\mathbf{X}_{t,o}$ to decide the final match.

The novelty of $\mathcal{M}_{\text{HER}}$ includes (a) an effective fusion layer for tuple-object alignment, and (b) a distillation-based fine-tuning strategy that trains the layer without human annotations.

Below we describe the details of each component of $\mathcal{M}_{\text{HER}}$.

Representation encoder $\mathcal{M}_{\text{rep}}$. We adopt a pre-trained multi-modal network (*e.g.,* [34]) as $\mathcal{M}_{\text{rep}}$, which serializes t (resp. o) following [35] and maps it into an embedding $\mathbf{t}$ (resp. $\mathbf{o}$).

Fusion layer $\mathcal{M}_{\text{fusion}}$. To facilitate in-depth analysis, a multi-modal fusion layer captures correlations across modalities.

Given $\mathbf{t}$ and $\mathbf{o}$ of modality m (m is text, image, audio or video), we first generate a joint feature representation [36]:

$$\mathbf{J}_m = \mathsf{FC}([\mathbf{t}; \mathbf{o}]) \in \mathbb{R}^{2d \times 1},$$

where FC is a fully connected layer with a learnable matrix.

Then we generate the fusion block. We set two learnable weight matrices (*i.e.,* the attention) $\mathbf{W}_m^t \in \mathbb{R}^{d \times 2d}$ and $\mathbf{W}_m^o \in \mathbb{R}^{d \times 2d}$ for each modality m and compute the cross-correlation matrix $\mathbf{C}_m^t$ (resp. $\mathbf{C}_m^o$) from $\mathbf{t}$ (resp. $\mathbf{o}$) to $\mathbf{J}_m$, as follows:

$$\mathbf{C}_m^t = \tanh(\frac{\mathbf{t}\mathbf{J}_m^\mathsf{T} \odot \mathbf{W}_m^t}{\sqrt{d}}), \qquad \mathbf{C}_m^o = \tanh(\frac{\mathbf{o}\mathbf{J}_m^\mathsf{T} \odot \mathbf{W}_m^o}{\sqrt{d}}),$$

where $\odot$ is element-wise multiplication. To better fuse multi-modal features, we compute two attention maps $\mathbf{h}_m^t$ and $\mathbf{h}_m^o$:

$$\mathbf{h}_m^t = \mathsf{ReLU}(\mathbf{C}_m^t \mathbf{W}_m^t \mathbf{t}), \qquad \mathbf{h}_m^o = \mathsf{ReLU}(\mathbf{C}_m^o \mathbf{W}_m^o \mathbf{o}).$$

Here $\mathbf{h}_m^t$ (resp. $\mathbf{h}_m^o$) captures feature importance in t (resp. o).

After two consecutive fusion blocks, we compute the final similarity representation between tuple t and object o to be:

$$\mathbf{X}_{t,o} = [\mathbf{X}_m^t; \mathbf{X}_m^o] \in \mathbb{R}^{2d \times 1},$$

where $\mathbf{X}_m^t$ (resp. $\mathbf{X}_m^o$) is the attended feature of tuple t (resp. object o), *i.e.,* $\mathbf{X}_m^t = \mathbf{W}_m^t \mathbf{h}_m^t + \mathbf{t}$ and $\mathbf{X}_m^o = \mathbf{W}_m^o \mathbf{h}_m^o + \mathbf{o}$.

Binary classifier $\mathcal{M}_{\mathsf{bin}}$. We transform $\mathbf{X}_{t,o}$ to a binary decision using a learnable matrix $\mathbf{W}_{\mathsf{bin}} \in \mathbb{R}^{2d \times 2}$ and the softmax activation function, *i.e.,* $\mathbf{p} = \mathsf{softmax}(\mathbf{X}_{t,o}^\mathsf{T} \mathbf{W}_{\mathsf{bin}}) \in \mathbb{R}^{1 \times 2}$. Here $\mathbf{W}_{\mathsf{bin}}$ is shared across all modalities to make sure that different modalities are mapped to the same latent space.

Training based on distillation. We fine-tune $\mathcal{M}_{\mathsf{HER}}$ by leveraging a large multi-modal VLM, such as QWen2.5-VL [20], as the labeler. However, directly labeling all tuple-object pairs from $(\mathcal{D}, \mathcal{U})$ is expensive and it incurs a quadratic complexity. To maintain effectiveness and label-efficiency, we propose a distillation-based training strategy, inspired by margin-based active learning [37]. Intuitively, we iteratively retrieve candidate matching pairs from $(\mathcal{D}, \mathcal{U})$ using $\mathcal{M}_{\mathsf{rep}}$, identify ambiguous ones via $\mathcal{M}_{\mathsf{HER}}$, and delegate those pairs to VLM for labeling, with which $\mathcal{M}_{\mathsf{HER}}$ is able to make more accurate decisions. This process repeats until a labeling budget is exhausted.

Complexity. The training takes $O(|\mathcal{T}_{\mathsf{train}}||\mathcal{M}_{\mathsf{HER}}|\mathsf{itr} + |\mathsf{VLM}|\mathcal{B})$ time, where $\mathcal{T}_{\mathsf{train}}$ is the set of training tuple-object pairs, itr is the number of iterations and $\mathcal{B}$ is the labeling budget of VLM.

V. Discovering DCRs in Parallel

This section states the discovery problem (Section V-A) and models discovery as a learning problem (Section V-B). It proposes learning-based discovery algorithm (Section V-C) and parallelizes it with parallel scalability [38] (Section V-D).

A. The Discovery Problem

We want "useful" DCRs with high support and confidence.

Support. The support of $\varphi = X \to p_0$ indicates how often φ could apply to $(\mathcal{D}, \mathcal{U})$, defined based on the types of p_0.

- [C1] p_0 *involves two variables, e.g.,* $\mathcal{M}_R(e_1, e_2)$ or $e_1.A \oplus e_2.B$. Following [29], the support of φ on $(\mathcal{D}, \mathcal{U})$, denoted as $\mathsf{supp}(\varphi, (\mathcal{D}, \mathcal{U}))$, or simply as $\mathsf{supp}(\varphi)$, is defined as:

  $|\{\langle h(e_1), h(e_2)\rangle \mid h \text{ is a valuation of } \varphi, h \models X, h \models p_0\}|,$

  i.e., the number of all pairs $\langle h(e_1), h(e_2)\rangle$ such that $h \models \varphi$.
- [C2] p_0 *involves only one variable, e.g.,* $e.A \oplus c$. We define the support of φ similarly, *i.e.,* $\mathsf{supp}(\varphi) =$

  $\gamma|\{h(e) \mid h \text{ is a valuation of } \varphi, h \models X \text{ and } h \models p_0\}|,$

  where γ is a scaling factor that scales up [C2] so that [C1] and [C2] are of the same scale. In practice, we set γ to be the maximum number of tuples in a relation in $\mathcal{D}$.

Given $\varphi = X \to p_0$ and $\varphi' = X' \to p_0$ with the same consequence p_0, we say that φ *subsumes* φ', denoted as $\varphi \preceq \varphi'$, if $X \subseteq X'$, *i.e.,* φ is more general than φ'. Along the same lines as in [29], one can verify that support has the *anti-monotonicity* property, *i.e.,* if $\varphi \preceq \varphi'$, then $\mathsf{supp}(\varphi) \geq \mathsf{supp}(\varphi')$.

For a positive integer σ, φ is σ*-frequent* if $\mathsf{supp}(\varphi) \geq \sigma$.

Confidence. The confidence of φ, denoted as $\mathsf{conf}(\varphi)$, indicates the reliability of φ, *i.e.,* how often φ is true if X is true:

$$\mathsf{conf}(\varphi) = \frac{\mathsf{supp}(\varphi)}{\mathsf{supp}(X)}, \text{ where } \varphi = X \to p_0.$$

Here $\mathsf{supp}(X)$ is the number of $\langle h(e_1), h(e_2)\rangle$ with $h \models X$ if p_0 has two variables e_1 and e_2; similarly for uni-variable p_0.

For a threshold δ, a DCR is δ*-confident* if $\mathsf{conf}(\varphi) \geq \delta$.

Minimality. A DCR $\varphi = X \to p_0$ is *minimal* if it is (a) *non-trivial*, *i.e.,* $p_0 \notin X$, and (b) *left-reduced*, *i.e.,* φ is σ-frequent and δ-confident, and no $\varphi' \preceq \varphi$ is σ-frequent and δ-confident.

Cover of DCRs. To remove redundant DCRs that are logical consequence of other DCRs, we define the *cover* Σ^c of a set Σ of DCRs to be a subset Σ^c of Σ such that (1) $\Sigma^c \models \Sigma$, *i.e.,* $\Sigma^c \models \varphi$ for all $\varphi \in \Sigma$, and (2) $\Sigma^c \setminus \{\varphi\} \not\models \Sigma^c$ for any $\varphi \in \Sigma^c$.

Discovery of DCRs. DCRs from $(\mathcal{D}, \mathcal{U})$ can be excessive and misaligned with users' interests [29], [39]. Following common practice, we (a) select a set $\mathcal{P}_0$ of application-dependent consequences, (b) select a set $\mathcal{P}_X$ of candidate predicates for constructing preconditions, and (c) fix a set $\mathcal{E}$ of entities, restricting discovery to DCRs that apply to at least one entity in $\mathcal{E}$.

Objective function. An objective function $f(\Sigma)$ measures the accuracy of a set Σ of DCRs. We adopt the F1-score for the error detection accuracy as the objective, although other objectives can also be plugged in without altering our methodology.

Given a set Σ of DCRs and a testing set $(\mathcal{D}_{\mathsf{test}}, \mathcal{U}_{\mathsf{test}})$, we define the objective $f(\Sigma)$ to be $\frac{2 \times \mathsf{pre} \times \mathsf{rec}}{\mathsf{pre} + \mathsf{rec}}$, where pre (resp. rec) is the ratio of correctly detected errors in $(\mathcal{D}_{\mathsf{test}}, \mathcal{U}_{\mathsf{test}})$ to all the violations of Σ (resp. all errors in $(\mathcal{D}_{\mathsf{test}}, \mathcal{U}_{\mathsf{test}})$).

Problem. The discovery problem of DCRs is stated as follows.

- *Input*: A mixed dataset $(\mathcal{D}, \mathcal{U})$, sets $\mathcal{P}_X$, $\mathcal{P}_0$ and $\mathcal{E}$ as above, a support threshold $\sigma > 0$, a confidence threshold $\delta > 0$, a positive integer η, and an objective function $f(\cdot)$.
- *Output*: A cover of the set Σ of DCRs such that $f(\Sigma)$ is maximized and for each $\varphi = X \to p_0$ in Σ, (a) $X \subseteq \mathcal{P}_X$ and $p_0 \in \mathcal{P}_0$, (b) φ is minimal, σ-frequent and δ-confident, (c) φ can be applied to at least one entity in $\mathcal{E}$, (d) $|X| \leq \eta$.

We denote by $\mathsf{req}(\mathcal{P}_X, \mathcal{P}_0, \sigma, \delta, \eta)$ the discovery requirement, abbreviated as req. A DCR φ is *valid* if it satisfies req. Here η controls the cost of discovery, and the use of $\mathcal{P}_X$, $\mathcal{P}_0$ and $\mathcal{E}$ makes the discovery focus on the users' needs; these can be set directly by users or computed automatically as in [29], [39].

B. Discovery as a Learning Problem

One might want to adapt existing relational rule-discovery algorithms to DCRs; however, this approach does not scale. Such methods often rely on exhaustive enumeration to produce complete rule sets, incurring exponential cost despite pruning strategies [39], [40]. As reported in [41], rule discovery over 60 predicates fails to terminate within 12 hours on a dataset of 1M tuples with 22 attributes. This cost is already prohibitive for purely relational data and becomes much worse for DCR discovery, whose search space spans relational data, unstructured objects, ML predicates and heterogeneous bindings, making exhaustive enumeration infeasible. Moreover, many discovered rules are of limited value, catching only homogeneous errors and contributing little to the discovery objective.

In light of this, we propose a learning-based discovery algorithm built on *Monte Carlo Tree Search (MCTS)*. We adopt MCTS because (a) it adaptively explores the vast search space, balancing exploration and exploitation, and has been successful in domains from game playing [42] to path discovery [43], [44]; (b) it can be enhanced with ML-guided simulations to guide the search toward high-value DCRs rather than attempting full enumeration; (c) it can be extended to return only DCRs that improve the objective; and (d) it is parallelizable and supports properties such as anytime discovery [43], making it suitable for large multi-modal data. This departs from existing rule discovery or pattern mining algorithms: its goal is to discover an optimized, compact set of rules, not the entire rule space via exhaustive enumeration.

Below we first formulate DCR discovery as a Markov Decision Process (MDP). Then we tackle the MDP by MCTS.

1) MDP formulation: The MDP models sequential decision making as a quintuple $(\mathcal{S}, \mathcal{A}, T, \mathbb{R}, \lambda)$ under *observation space* $\mathcal{O}$, where (a) $\mathcal{S}$ is the *state* space, and each state $s \in \mathcal{S}$ captures the information for decision making; (b) $\mathcal{A}$ is the *action* space, and each action $a \in \mathcal{A}$ is a decision that affects the state; (c) T is the *transition probability* that describes the change from one state to another by taking an action; (d) $\mathbb{R}$ is the *reward function*, where a reward is associated with each transition, indicating its effect; and (e) λ is the discount factor.

Following existing discovery paradigm, we generate DCRs top-down, *i.e.,* for each consequence $p_0 \in \mathcal{P}_0$, we start with an empty precondition X and gradually expand it by adding predicates one at a time. This can be formulated as a MDP.

Observation $\mathcal{O}$. At timestamp t, an observation $o_t \in \mathcal{O}$ is p_0 and environment $(\mathcal{D}, \mathcal{U})$. It computes the support/confidence.

MDP: State and action. Let the current (partial) $\varphi = X \rightarrow p_0$ be a state s. Each $p \in \mathcal{P}_X$ to be added to X forms an action a. Denote the set of all actions that can be added to s by $\mathcal{A}(s)$.

We say that a state s is a *terminal state* (at which we take no further actions) if it is one of the following cases: (a) current φ is a valid DCR, (b) the support of φ is already below σ; by the anti-monotonicity, adding more predicates to X will further lower the support, leading to invalid DCRs, or (c) there is no more action to take (*i.e.,* $\mathcal{A}(s) = \emptyset$) or X already has η predicates. To decide whether s is a terminal state, we interact with the environment to get its support and confidence.

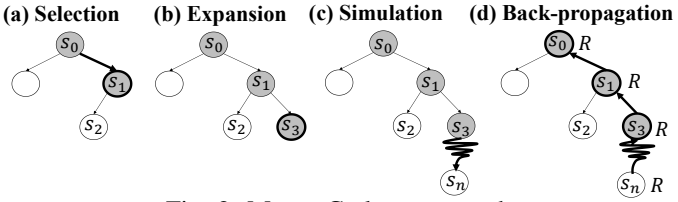


Fig. 2: Monte Carlo tree search

MDP: Transition. Consider a state s_t (*i.e.,* $\varphi_t = X_t \rightarrow p_0$) at timestamp t. If an action a (*i.e.,* a predicate p) in $\mathcal{A}(s_t)$ is selected, we *transit* to a new state s_{t+1} (*i.e.,* $\varphi_{t+1} = X_t \cup \{p\} \rightarrow p_0$), by simply adding p to X_t. Since $\mathcal{P}_X$ is known, the transition probability, *i.e.,* $T(s_{t+1}|s_t, a)$, is deterministic.

MDP: Reward. We define the rewards of states using a sparse reward function $\mathbb{R}()$. Specifically, if s_{t+1} is a terminal state whose φ_{t+1} is valid (*i.e.,* case (a)), $\mathbb{R}(s_{t+1})$ is 1. Otherwise, $\mathbb{R}(s_{t+1})$ is 0 for other terminal states (*i.e.,* cases (b) & (c)) and intermediate states. Since we transit to s_{t+1} by taking action a at state s_t, we also define the reward for such action-state pair, denoted by $\mathbb{R}(a, s_t)$, to be $\mathbb{R}(a, s_t) = \mathbb{R}(s_{t+1})$.

2) MCTS: To explore the search space, MCTS iteratively builds a search tree of states. Each node (resp. edge) represents a state (resp. an action), and we use node/state and edge/action interchangeably. The root corresponds to the initial state with an empty precondition X. A child node is obtained by applying an action to its parent state, while a leaf node is either a terminal state or a newly expanded, unvisited state.

Overview. To guide the search, MCTS iterates until it reaches the maximum number of iterations or the search space has been fully explored. Each iteration of MCTS can be divided into *selection*, *expansion*, *simulation* and *back-propagation*.

- *Selection*. A node s_t is selected from the search tree. If s_t is terminal, we proceed with back-propagation. Otherwise, s_t is not fully expanded and we move to the expansion step.
- *Expansion*. Given selected s_t, the most promising child s_{t+1} is added to the tree according to actions in $\mathcal{A}(s_t)$.
- *Simulation*. For the newly added node s_{t+1}, a simulation is conducted at s_{t+1}, by taking a sequence of actions based on a fast rollout policy, until a terminal state is reached.
- *Back-propagation*. The reward of the terminal state is computed and it is back-propagated bottom up to the root.

Finally, the set Σ of valid DCRs stored in all leaf nodes of the search tree is collected and its cover Σ^c is returned.

Note that we employ two ML models to warrant efficient and effective MCTS: (a) a policy model $\mathcal{M}_p$ that is learned to select a good action that most likely leads to valid DCRs, and (b) a value model $\mathcal{M}_v$ that facilitates the reward computation. We postpone the details of these two models to Section V-C.

Example 3: An example MCTS iteration is shown in Figure 2. Starting from the root s_0, we traverse the search tree and select the node s_1. Since s_1 is not fully expanded, we expand it by adding a new child s_3 into the tree. We then run a simulation at

s_3 and reach a terminal state s_n. The reward of s_n is computed based on $\mathbb{R}()$ and back-propagated from s_n to the root. □

Below we detail the four steps of an MCTS iteration.

Selection. Starting from the root, MCTS traverses the tree by recursively decide the next child node to which we move. When it ends, it reaches the selected node s_t which is either a terminal node or an intermediate node not fully expanded. The selection aims to balance exploration and exploitation, to promote unexplored areas and stick to promising ones.

To achieve this, we maintain the following statistics for each node s_t: (a) the number $N(s_t)$ of times that node s_t has been visited, and (b) the expected reward $Q(s_t)$ of node s_t, *i.e.,* the proportion of valid DCRs found among all simulations walked through s_t. Suppose that we transit to s_{t+1} by taking action a at state s_t. We similarly maintain (c) the number $N(s_t, a)$ of times that an edge a has been visited, and (d) the expected reward $Q(s_t, a)$ by taking action a at state s_t, where we set $Q(s_t, a) = Q(s_{t+1})$ and $N(s_t, a) = N(s_{t+1})$.

We initialize $N(s_t)$ to 0 for all states s_t. For the reward $Q(s_t)$, it consists of two parts: (a) an empirical value $\hat{Q}(s_t)$ estimated through MCTS back-propagation (initialized to 0), and (b) a value $Q_v(s_t)$ estimated by the value model $\mathcal{M}_v$, *i.e.,*

$$Q(s_t) = \alpha \cdot \hat{Q}(s_t) + (1 - \alpha) \cdot Q_v(s_t),$$

where α is a balancing parameter. Intuitively, this enables effective exploration guidance from supervised models.

MCTS adopts the *Upper Confidence Bound (UCB)* as the score function to decide the next action a to take as follows:

$$a = \arg\ \max_{a \in \mathcal{A}(s_t)} \left(Q(s_t, a) + c \cdot \sqrt{\frac{\ln\ N(s_t)}{N(s_t, a)}} \right),$$

where c is a hyper-parameter to consider the trade-off between the exploitation of promising actions (the first term) and the exploration of unseen search space (the second term).

Expansion. If the selected s_t is a terminal state, we directly compute its $\mathbb{R}(s_t)$. Otherwise, s_t is not fully expanded and we pick the most promising action a from $\mathcal{A}(s_t)$, transit from s_t to s_{t+1} and add node s_{t+1} to the tree. Specifically, we select action a such that $a = \arg\ \max_{a \in \mathcal{A}(s_t)} Q(s_t, a)$.

Note that the search space of DCR discovery is a huge "lattice" and worse still, there can be redundant nodes, since the precondition of a DCR is *permutation-invariant* [39]: the same DCR can be generated from different branches of the tree, by adding predicates in different orders. This incurs inaccuracy, *e.g.,* a node s may be visited several times but its visit count $N(s)$ will not contribute to other nodes for the same DCR.

To reduce the search space and avoid redundant nodes in the tree, we adopt the two strategies to restrict the action space.

(1) Prioritized expansion. We classify actions into different priorities, *e.g.,* priorities from high to low are (a) binding predicates, (b) predicates on relational data, (c) HER predicates, and (d) predicates across relational and unstructured data.

(2) Predicate ordering. We enforce a total ordering Ω (*e.g.,* the lexicographic ordering) on actions in the same priority level.

Consider a state s_t transited from s_{t-1} by taking action a_{t-1}. We restrict $\mathcal{A}(s_t)$ to be the set of all actions a_t such that either (a) the priority of a_t is lower than that of a_{t-1} or (b) a_{t-1} and a_t are of the same priority level, but a_t is placed after a_{t-1} in Ω. In this way, there is no redundant node in the tree and we can explore the search space more efficiently.

Simulation. From the newly added s_{t+1}, we run a fast simulation to reach a terminal state. Rather than random actions [43], which poorly capture the potential of s_{t+1} to yield valid DCRs, we use a policy model $\mathcal{M}_p$ to guide fast rollouts (Section V-C), and compute the reward $\mathbb{R}(s_n)$ at the terminal state s_n.

Note that the states visited during simulation are not memorized, *i.e.,* they are not stored as tree nodes. This can be wasteful: even if a terminal state s_n corresponds to a valid DCR, it may be missed in Σ, which is collected only from tree leaves, if the search tree is not sufficiently expanded. To address this, we maintain a memory set $\mathbb{M}$. Whenever a valid DCR φ is encountered during simulation, it is added to $\mathbb{M}$; upon termination, the DCRs in $\mathbb{M}$ are also returned.

Back-propagation. We update the statistics maintained in the tree (*i.e.,* $\hat{Q}()$ and $N()$), by traversing from the terminal state s_n (a leaf or a simulated node) back to the root. Specifically, for each tree node s_i visited in this MCTS iteration,

$$\hat{Q}(s_i) = \frac{\hat{Q}(s_i)N(s_i) + \mathbb{R}(s_n)}{N(s_i) + 1} \text{ and } N(s_i) = N(s_i) + 1,$$

where $\mathbb{R}(s_n)$ is the reward of the terminal state s_n. Intuitively, by this mechanism, the number of visits is always incremented by one and the expected reward is recursively refined.

Example 4: Consider s_1 in Figure 2. Assume in the previous iteration, s_2 is just added, whose simulation leads to an invalid DCR with reward 0, yielding $N(s_1) = 1$ and $\hat{Q}(s_1) = 0$ (since the only simulation through s_1 has 0 reward). Assume that at this iteration (Figure 2 (d)), $\mathbb{R}(s_n) = 1$. We update $N(s_1)$ and $\hat{Q}(s_1)$ to be $1 + 1 = 2$ and $\frac{0 \times 1 + 1}{1 + 1} = \frac{1}{2}$, respectively. □

Remark. MCTS supports *anytime discovery* [43], *i.e.,* it can be stopped anytime when the maximum exploration budget (*e.g.,* the maximum number of iterations) is reached and a solution is always available; the solution is gradually improved with time and it converges to the optimal solution (*i.e.,* it finds all valid DCRs) if given enough budget.

C. A MCTS-Guided Discovery Algorithm

Utilizing MCTS, we develop a learning-based discovery algorithm, DCRLeaner, in Figure 3. Given a mixed dataset $(\mathcal{D}, \mathcal{U})$, the requirement req and the bound $I_{\max}$ of MCTS iterations, DCRLeaner returns a cover set Σ^c of valid DCRs.

DCRLeaner first pre-computes the predictions of ML predicates in $\mathcal{P}_X$, and then trains the policy model $\mathcal{M}_p$ and the value model $\mathcal{M}_v$, used for evaluating the actions and states, respectively (line 1-2, see below). Then DCRLeaner processes each consequence p_0 in $\mathcal{P}_0$ one by one (lines 3-13).

For each p_0, we build a search tree in $I_{\max}$ iterations, initializing the root node as $s_0 = \emptyset \rightarrow p_0$ (line 4). For simplicity, the tree is denoted by its root s_0. In each MCTS

Input: $(\mathcal{D},\mathcal{U})$, $\mathsf{req}(\mathcal{P}_X,\mathcal{P}_0,\sigma,\delta,\eta)$, objective $f(\cdot)$ and a bound I_{max}.
Output: A cover Σ^c of DCRs conforming to $\mathsf{req}(\mathcal{P}_X,\mathcal{P}_0,\sigma,\delta,\eta)$.

1. Pre-compute the predictions of ML predicates in $\mathcal{P}_X$;
2. $\Sigma := \emptyset$; $\mathbb{M} := \emptyset$; Train policy model $\mathcal{M}_p$ and value model $\mathcal{M}_v$;
3. **for** each p_0 in $\mathcal{P}_0$ **do**
4. $\quad s_0 := \emptyset \rightarrow p_0$; $\mathsf{itr} := 0$; Initialize the search tree with root s_0;
5. $\quad$ **while** $\mathsf{itr} < I_{\mathsf{max}}$ **do**
6. $\quad\quad s_t := \mathsf{selection}(s_0)$; $s_n := \mathsf{null}$;
7. $\quad\quad$ **if** s_t is a terminal state **then**
8. $\quad\quad\quad s_n := s_t$;
9. $\quad\quad$ **else** $s_{t+1} := \mathsf{expansion}(s_t)$;
10. $\quad\quad\quad$ Add s_{t+1} as the child node of s_t into the tree;
11. $\quad\quad\quad s_n := \mathsf{simulation}(s_{t+1},\mathcal{M}_p)$;
12. $\quad\quad\quad$ **if** the DCR φ_n of s_n is valid **then** $\mathbb{M} := \mathbb{M} \cup \{\varphi_n\}$;
13. $\quad\quad s_0 := \mathsf{back\text{-}propagation}\ (s_n,\mathcal{M}_v)$; $\mathsf{itr} := \mathsf{itr} + 1$;
14. $\quad$ **for** each valid φ in the leaf of the search tree s_0 or in $\mathbb{M}$ **do**
15. $\quad\quad$ **if** $f(\cdot)$ is enhanced by including φ **then**
16. $\quad\quad\quad \Sigma := \Sigma \cup \{\varphi\}$;
17. **return** $\mathsf{getCover}(\Sigma)$;

Fig. 3: Learning-based discovery algorithm DCRLeaner

iteration, we first select a node s_t in the tree via procedure selection using the UCB (Equation (V-B2)), so that s_t is either a terminal state or a not-fully-expanded intermediate node. If s_t is a terminal state (lines 7-8), we set it as s_n, *i.e.,* the node where the back-propagation starts. Otherwise, we expand s_t by adding a node s_{t+1} via procedure expansion (lines 9-10), and then call procedure simulation at s_{t+1} under policy model $\mathcal{M}_p$, to reach the simulated terminal state s_n (line 11). If the DCR φ_n of s_n is a valid DCR, it is added to the memory $\mathbb{M}$ (line 12). Finally, we back-propagate the visit count $N()$ and the empirical reward $\hat{Q}()$ from s_n to the root s_0 via procedure back-propagation (line 13, Equation (V-B2)). The empirical $\hat{Q}()$ is combined with the value estimated from value model $\mathcal{M}_v$ to get the final expected reward $Q()$ (Equation (V-B2)).

When the MCTS for each p_0 ends, we process each valid φ in the leaves or in memory $\mathbb{M}$ (lines 14-16): if the objective function $f(\cdot)$ is enhanced by including φ, we add φ to Σ.

Finally, we compute the cover of Σ via implication analysis, by adapting the efficient heuristic of [39] to DCRs (line 17).

Example 5: We illustrate MCTS for $p_0 : t.\mathsf{pid} \neq t'.\mathsf{pid}$ in Figure 2 and show how φ_1 is found. Initially, the root s_0 is $\emptyset \rightarrow p_0$. In the first MCTS iteration, $p_{\mathcal{M}} : \mathcal{M}_{\mathsf{ER}}(t,t')$ is added to X, leading to a new state s_1, *i.e.,* $\varphi_1' : p_{\mathcal{M}} \rightarrow p_0$. Since s_1 is not terminal, a simulation is run at s_1, followed by back-propagation. In the second MCTS round, the newly expanded s_1 is picked by the UCB. The most promising child s_2 of s_1 is added to the tree, by taking the action $t'.\mathsf{allergy} =$ Penicillin, followed by simulation and back-propagation from s_2. After several MCTS iterations, a terminal state s_n whose associated DCR is φ_1 (Example 2) is added to the tree. When the MCTS ends, φ_1 is collected and returned in Σ. □

Pre-processing of predicates. Following the common practice of rule discovery for ML-augmented rules [29], [45], we pre-evaluate the ML predicates and the HER operator $\Theta_{\asymp}(t,o)$ in $\mathcal{P}_X$, materializing their results prior to the discovery:

- $\mathcal{M}_U(\mathcal{U},t)$. Depending on the type of unstructured data $\mathcal{U}$ (*e.g.,* documents, images, audio, video), we pre-evaluate $\mathcal{M}_U(\mathcal{U},t)$ for each tuple t using the corresponding $\mathcal{M}_U$ (see Section II-A), and store the extracted objects o for t.
- $\Theta_{\asymp}(t,o)$. We use a filter-and-verify method: representation encoder $\mathcal{M}_{\mathsf{rep}}$ retrieves candidate tuple-object pairs from $(\mathcal{D},\mathcal{U})$ and distillation-based model $\mathcal{M}_{\mathsf{HER}}$ verifies them. The verified matches are stored as key-value pairs, where tuple IDs serve as keys and matched object IDs as values.
- $\mathcal{M}_R(e_1,e_2)$. Similar to $\Theta_{\asymp}(t,o)$, we train a lightweight representation model $\mathcal{M}^e_{\mathsf{rep}}$ that learns the embeddings of e_1 and e_2, via distillation-based training, except that the labeler becomes $\mathcal{M}_R(e_1,e_2)$ instead of VLM. We only fine-tune $\mathcal{M}^e_{\mathsf{rep}}$ using the labeled data. During inference, we first use $\mathcal{M}^e_{\mathsf{rep}}$ to retrieve a set of potential matching pairs (e_1,e_2) and then apply $\mathcal{M}_R$ to make the final prediction. The matching results are also stored in KV-pairs.

Value model $\mathcal{M}_v$ and policy model $\mathcal{M}_p$. To reduce the training overhead, we instantiate $\mathcal{M}_v$ and $\mathcal{M}_p$ with the same lightweight regression backbone (*i.e.,* random forest regression and MLP), where $\mathcal{M}_v$ estimates the rewards and $\mathcal{M}_p$ rolls out by iteratively selecting the action with the highest reward.

The pipeline works as follows. (1) We first run standard MCTS on small sampled data to obtain empirical $\hat{Q}(s)$ for each visited state s. (2) For each s, we encode its statistic (*e.g.,* the number of selected predicates, support, and confidence, etc) into a feature vector $\mathbf{v}_s$. (3) We train $\mathcal{M}_v$ and $\mathcal{M}_p$ on paired data $\{(\mathbf{v}_s,\hat{Q}(s))\}$. Once the training converges, the two models are frozen and used as the value and policy models.

Complexity. DCRLeaner takes $O(c_{\mathsf{valid}} I_{\mathsf{max}} F|\mathcal{P}_0||P_X|)$ time, where F is the tree depth and c_{valid} denotes the unit cost of rule validation on $(\mathcal{D},\mathcal{U})$, since for each p_0 in $\mathcal{P}_0$, it performs selection, expansion, simulation and back-propagation for $O(I_{\mathsf{max}})$ MCTS iterations, where the four steps take $O(F|P_X|)$, $O(c_{\mathsf{valid}}+|P_X|)$, $O(c_{\mathsf{valid}}F|P_X|)$ and $O(F)$ time, respectively.

D. A Parallel Discovery Algorithm

We parallelize DCRLeaner to PDCRLeaner in the same way as its counterpart for REEs [29], [39]. PDCRLeaner runs with one coordinator and n workers under the Bulk Synchronous Parallel (BSP) model [46], where the coordinator distributes and balances workloads, and the workers run DCRLeaner in parallel. Given the set $\mathcal{P}_0$ of consequences, the coordinator evenly divides it into n partitions and constructs a set of independent tasks based on each partition. Each worker then handles the assigned tasks from a distinct partition.

Along the same lines as [29], [39], we say that PDCRLeaner is *parallelly scalable relative to* the sequential DCRLeaner if its runtime by using n processors can be expressed as:

$$T(|\mathcal{D}|,|\mathcal{U}|,\mathsf{req}) = \widetilde{\mathcal{O}}(\frac{t(|\mathcal{D}|,|\mathcal{U}|,\mathsf{req})}{n}),$$

where $t(|\mathcal{D}|,|\mathcal{U}|,\mathsf{req})$ is the worst runtime of sequential DCRLeaner, and $\widetilde{\mathcal{O}}()$ hides $\log(n)$ factors. Intuitively, parallel scalability guarantees "linear" speedup relative to "yardstick" DCRLeaner. That is, PDCRLeaner is able to scale with large

datasets by adding more processors when needed.

One can verify the following (see [15] for a proof).

Theorem 2: *Algorithm* PDCRLeaner *is parallelly scalable relative to the sequential algorithm* DCRLeaner. □

VI. Experimental Study

Using real-life data, we conducted experiments to evaluate the efficiency and effectiveness of (a) error detection (ED) with DCRs, and (b) the learning-based rule discovery (RD).

Experimental setting. We start with the setting.

Datasets. We used six datasets, including four benchmarks [14], one healthcare dataset, and one dataset from our industry partners (Table II): (1) Fakeddit: 21.9K instances of images, texts and relational data on fake news; (2) Goodreads: 16K of book covers, description and metadata; (3) Amazon: 25.8K of product images, description and metadata; (4) Posterlens: 25K of movie posters and metadata; (5) PAD: a specialized healthcare dataset comprising 2,298 skin-lesion images from 1,373 Brazilian patients, each accompanied by 21 numerical, categorical and textual tabular features [11]; (6) Bank: 5.01M tuples in 3 relations synthesized from public data, transactions, client info and credit card details, plus crawled knowledge and CBRC regulatory documents (PDF with text, tables, images).

We separated unstructured data from structured data to apply distillation-based HER. All datasets except Bank provide ground-truth links between relational tuples and their unstructured snippets. We use them directly as HER labels. For Bank, we only use a PDF if it is semantically aligned with the relational attributes. We randomly sampled 20K tuple-object pairs for training/testing/validation of HER with ratio 4:5:1.

We also partitioned each dataset into $\mathcal{D}_{\mathsf{train}}/\mathcal{D}_{\mathsf{test}}/\mathcal{D}_{\mathsf{valid}}$ for training/testing/validation with ratio 8:1:1. We trained models and learned DCRs on $\mathcal{D}_{\mathsf{train}}$, and evaluated (resp. validated) them on $\mathcal{D}_{\mathsf{test}}$ (resp. $\mathcal{D}_{\mathsf{valid}}$). Following standard practice, we treated the original values in $\mathcal{D}_{\mathsf{test}}$ as clean ground truth and randomly injected 10% errors into it to evaluate the performance of ED using the learned DCRs. The accuracy of ED is measured by F1-score (see Section V-A).

Baselines. We implemented PDCRLeaner in Python and compared: (1) HyperLogic [47], a framework that integrates hypernetworks into rule learning. (2) VLM [20], a 7B Visual Language Model based on Qwen-2.5-VL, further trained using LoRA and fine-tuned on validation data. (3) REEFinder [29], a traditional rule mining method based on levelwise search; we compared it for RD efficiency only. (4) OHunt [48], a meta-learning rule discovery method for outlier detection. Note that HyperLogic, REEFinder and OHunt are designed for relational data. To adapt them for cross-domain discovery, we (a) adopted the same pre-processing strategy as PDCRLeaner (Section V-C) to support ML predicates and HER operators, and (b) converted multi-modal inputs into relational form for these methods to mine multi-modal rules. For VLM, we serialize each multi-modal record into a single sequence using the ground-truth links and feed this sequence to the model.

For HER, we tested: (a) LLaVA-NeXT [49], a multi-modal LLM, which integrates an image encoder, LLM and a vision-language MLP projector, and (b) BLIP [50], a pretrained MED (Multimodal Encoder-Decoder) model for flexible vision-language understanding and generation. We used BLIP as an embedding model, and combined it with our fusion layer.

Datasets	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{R}\rvert$	$\lvert\mathcal{U}\rvert$	Modalities	$\lvert\Sigma\rvert$
Fakeddit [14]	21.9K	5	21.9K	{relations, text, image}	86
Goodreads [14]	16K	4	16K	{relations, text, image}	86
Amazon [51]	25.8K	24	25.8K	{relations, text, image }	62
Posterlens [52]	25K	5	25K	{relations, text, image }	44
PAD [11]	25K	22	2.3K	{relations, text, image }	480
Bank	5.01M	92	259K tokens	{relations, pdf file }	179

TABLE II: The tested datasets

Embedded ML models. We used the following pre-trained ML models in DCRs. (1) $\mathcal{M}_U$. For documents, we adopted Bert-MRC [18] and Bert [19] to extract entities and key-value pairs. For image, video and audio, we used the mult-modal LLM Qwen-2.5-VL [20] and BEATS [17]. (2) $\mathcal{M}_R$. We adopted the MM-Embed model [34] for all modalities, where we transform multi-modal data into embeddings, based on which we compute the similarities. The PDF file of Bank contains both attribute-level constraints on the relations and the regulatory business logic; we therefore feed it directly to Qwen-2.5-7B-instruct LLM to generate the ML predicates of DCRs.

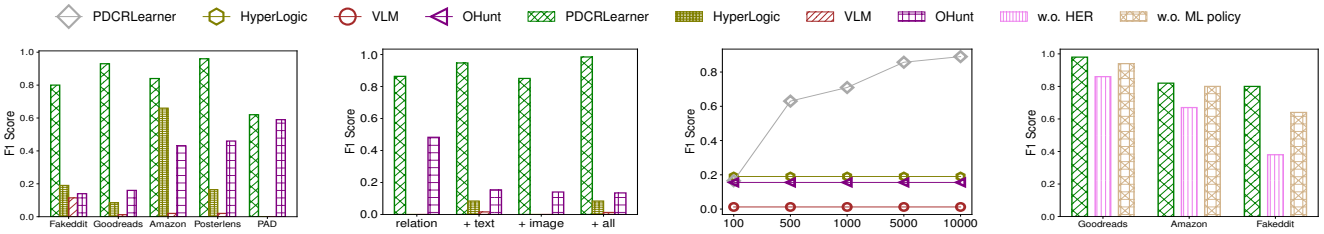
Parameter settings. By default, for $\mathcal{M}_{\mathsf{HER}}$, the label budget $\mathcal{B}$ is 8K, the learning rate is 1e-4 and the number of epochs is 100. For DCR discovery, the support threshold σ is $10^{-6}|\mathcal{D}|^2$, the confidence threshold δ is 0.7, the maximum number η of predicates (resp. the bound I_{max} of MCTS iterations) is 12 (resp. 500) for Bank and 6 (resp. 10,000) for others. The objective function $f(\Sigma)$ is the F1-score, the balancing parameter α is 0.4, the trade-off parameter c is 1.4, and the number n of processors is 20. The number of DCRs mined under default settings is summarized in Table II.

Configuration. We ran experiments on a machine with 504GB RAM and 104 Intel(R) Xeon(R) Gold 5320 CPUs @2.20GHz. Each experiment was run 3 times and the average is reported.

Experimental results. We next report our findings.

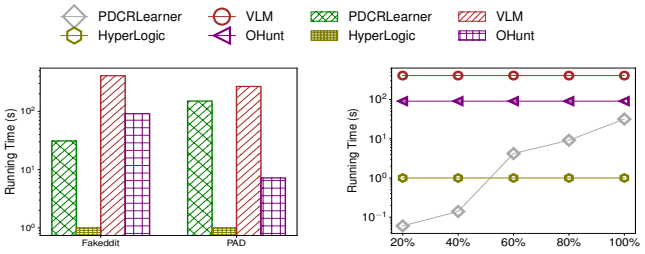
Exp-1: Effectiveness of ED. To evaluate the effectiveness of different methods, we detect errors on $\mathcal{D}_{\mathsf{test}}$: (a) for all rule-based methods (*i.e.,* HyperLogic, OHunt and PDCRLeaner), we detect errors as the violations of rules learned, and (b) for VLM, we directly use it to detect errors in $\mathcal{D}_{\mathsf{test}}$ by handcrafted prompts. A case study on PAD was reported in Example 1.

Accuracy. We first report the accuracy of all methods in Figure 4(a). PDCRLeaner is more effective than VLM, OHunt and HyperLogic, *e.g.,* its average accuracy (F1-score) is 0.79, 0.47 and 0.61 higher than the three baselines, respectively, up to 0.94, 0.77 and 0.84. VLM fails to achieve high accuracy since (a) it uses LoRA instead of full weight tuning, and (b) it does not leverage the correlation between different domains for ED. HyperLogic encounters a similar issue. OHunt is not accurate since it does not integrate ML predicates well in its meta-learning-based rule discovery process; as a consequence, the ML predicates are not utilized well in error detection.



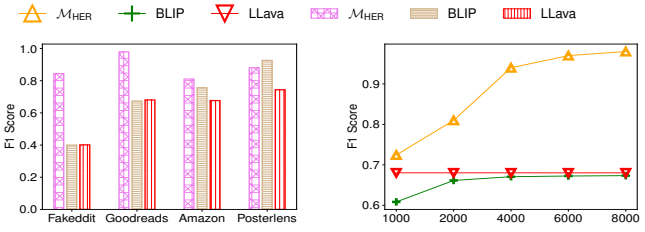
(a) ED: Accuracy (b) Goodreads: #modalities (c) Goodreads: Varying I_{max} (d) Ablation study

Fig. 4: Effectiveness of ED



(a) ED: Efficiency (b) Fakeddit: Varying $|\Sigma|$

Fig. 5: Efficiency of ED



(a) HER: Accuracy (b) Goodreads: Varying $\mathcal{B}$

Fig. 6: Effectiveness of HER

Varying #modalities. We started with structured data and report the accuracy by adding text only (+text), image only (+image), and all modalities (+all). As shown in Figure 4(b), the accuracy of PDCRLeaner increases more substantially than the baselines, especially after text is added, demonstrating its ability to effectively integrate multi-modal information. However, we observe that more modalities as additional features do not necessarily improve accuracy on Goodreads due to the noise introduced by multi-modal models, *e.g.,* adding images even reduces accuracy for all methods. Nevertheless, compared with other methods, PDCRLeaner is more robust to such noise, *e.g.,* its F1 only drops by 0.01, while the F1 of OHunt drops by 0.34. A more detailed evaluation about the impact of ML accuracy on PDCRLeaner is presented in Exp-5.

Varying I_{max}. We varied the bound I_{max} of MCTS iterations in Figure 4(c). PDCRLeaner exhibits increasing accuracy as I_{max} grows, since more iterations enable broader exploration and a higher likelihood of finding valid DCRs for ED. When I_{max} reaches 10,000, the accuracy gain slows, indicating that current exploration already yields sufficiently effective DCRs.

Varying c. Varying the trade-off parameter c from 0.1 to 3.0, we tested the accuracy of PDCRLeaner (not shown). When c is too small (resp. large), PDCRLeaner cannot find enough DCRs by over-exploiting (resp. over-exploring). When $c = 1.4$, PDCRLeaner strikes a good balance between the exploitation and the exploration, achieving the highest accuracy.

Varying α. We also varied the parameter α from 0.1 to 1.0 (not shown); the accuracy fluctuates and peaks at $\alpha = 0.4$.

Exp-2: Efficiency of ED. We report the time for ED on $\mathcal{D}_{\text{test}}$: for rule-based methods, we report the rule execution time for finding violations, and for VLM, we report its inference time.

Efficiency. As shown in Figure 5(a), PDCRLeaner runs comparably with most baselines, *e.g.,* it is 13.2× and 2.9× faster than VLM and OHunt on Fakeddit. HyperLogic is a pure ML model that efficiently uses matrix operations, and is the fastest. On PAD, PDCRLeaner needs more rules to catch more errors, yielding longer time. Given the accuracy gain of PDCRLeaner on all datasets (Exp-1), its execution time is acceptable.

Varying Σ. We studied the impact of the number $|\Sigma|$ of rules for ED, varying it from 20% to 100% in Figure 5(b). As expected, PDCRLeaner takes longer given more rules, while the running time of other baselines is not affected by $|\Sigma|$.

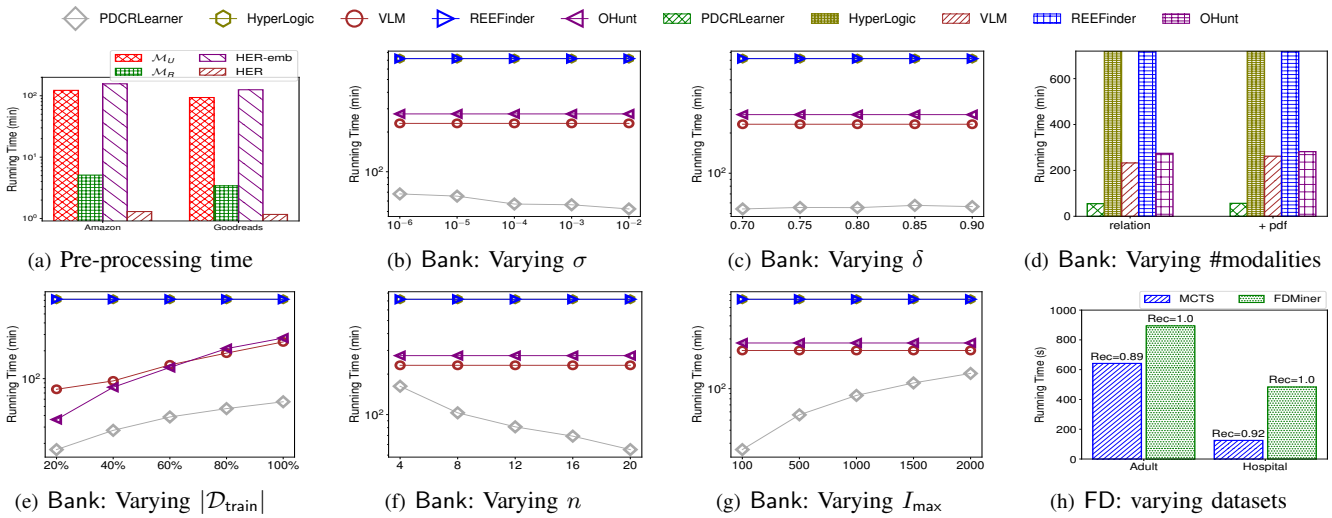
Exp-3: Effectiveness of $\mathcal{M}_{\text{HER}}$. We tested $\mathcal{M}_{\text{HER}}$'s accuracy by $\text{F1} = 2 \times \frac{\text{rec} \times \text{pre}}{\text{rec} + \text{pre}}$, where pre (resp. rec) is the ratio of correctly matched tuple-object pairs to all matched pairs (resp. true matches). We omitted PAD whose images provide only auxiliary information with no overlapping features and directly used the predefined mappings without training an HER model.

Accuracy. As shown in Figure 6(a), $\mathcal{M}_{\text{HER}}$ beats LLaVA-NeXT and BLIP (except on Posterlens) by 43% and 36% on average, respectively, up to 110% and 111%, verifying the usefulness of our fusion and distillation strategies. In contrast, BLIP's similarity-based representations struggle to capture true matches, while LLaVA-NeXT lacks a stable prompting method for different scenarios. Note that on Posterlens, $\mathcal{M}_{\text{HER}}$ is slightly less accurate than BLIP since $\mathcal{M}_{\text{HER}}$ is only trained on a single movie domain, whereas BLIP leverages large scale pre-training across diverse domains, equipping it with broader knowledge for distinguishing movie-related entities.

Varying $\mathcal{B}$. We studied the impact of labeling budget $\mathcal{B}$ by varying it from 1K to 8K. As shown in Figure 6(b), the accuracy of most methods improves when $\mathcal{B}$ gets larger, since more training data generally leads to more accurate models. Notably, $\mathcal{M}_{\text{HER}}$ shows a faster accuracy gain, benefiting from distillation training that selects more informative instances.

Exp-4: Efficiency of RD. We next compared the RD time of PDCRLeaner with the training or discovery time of baselines.

Pre-processing time. We first report time breakdown of predi-



(a) Pre-processing time (b) Bank: Varying σ (c) Bank: Varying δ (d) Bank: Varying #modalities

(e) Bank: Varying $|\mathcal{D}_{\text{train}}|$ (f) Bank: Varying n (g) Bank: Varying I_{max} (h) FD: varying datasets

Fig. 7: Efficiency of RD

cate pre-processing in Figure 7(a). On average, pre-computing predictions for ML predicates and HER operators (including HER embedding and HER model) takes 4.2 hours. Recall that all rule-based methods adopt the same pre-processing strategy as PDCRLeaner to enable cross-domain discovery. All such methods incur identical pre-processing costs; therefore we exclude that time and focus only on the discovery time.

In the rest of efficiency tests, we concentrate on the largest dataset Bank. When a method cannot finish in 10 hours on Bank, we show its running time as 10 hours in the figures.

Varying σ. We varied the support threshold σ from $10^{-6}|\mathcal{D}|^2$ to $10^{-2}|\mathcal{D}|^2$. As shown in Figure 7(b), PDCRLeaner exploits the anti-monotonicity of support and thus, has faster running time when σ increases. Moreover, PDCRLeaner is $6.8\times$ faster than VLM. This efficiency stems from its policy model $\mathcal{M}_p$, which effectively guides the discovery toward optimized rules. Since REEFinder enumerates all predicate combinations levelwise, it cannot finish in 10 hours. Similarly, HyperLogic run out of time. OHunt needs 4.9h for discovery via meta learning.

Varying δ. As shown in Figure 7(c), we tested the discovery time by varying the confidence δ from 0.7 to 0.9. As confidence is not anti-monotonic, PDCRLeaner gets slightly faster given smaller δ, as less rules are executed and terminated early.

Varying #modalities. As shown in Figure 7(d), after adding PDFs to relations, PDCRLeaner runs $1.1\times$ slower, yet remains $4.4\times$ and $5.6\times$ faster than VLM and OHunt, respectively, showing that learning-based rule discovery outperforms purely ML approaches. HyperLogic does not finish within 10 hours.

Varying $\mathcal{D}_{\text{train}}$. In Figure 7(e), we varied $|\mathcal{D}_{\text{train}}|$ from 20% to 100%. All methods take longer given more training data, *e.g.,* our method is $2.7\times$ slower when $|\mathcal{D}_{\text{train}}|$ is from 20% to 100%.

Varying n. We varied the number n of processors in Figure 7(f), PDCRLeaner is $2.5\times$ faster when n increases from 4 to 20. The speedup is modest because PDCRLeaner also trains an ML policy on a single GPU to identify higher-value DCRs.

Varying I_{max}. In Figure 7(g), we varied the value of I_{max} from 100 to 2000. PDCRLeaner exhibits nearly linear slowdown as I_{max} increases, since it decides the number of MCTS iterations.

	x=0	x=5	x=10	x=15
$\mathcal{M}_U$	0.589/0.645/0.634	0.571/0.696/0.627	0.598/0.643/0.619	0.633/0.637/0.635
$\mathcal{M}_R$	0.589/0.645/0.634	0.608/0.630/0.619	0.620/0.615/0.618	0.569/0.659/0.611

TABLE III: Impact of ML accuracy (precision/recall/F1)

Adaptation to FD discovery. To validate the MCTS strategy, we transplant it to functional dependency (FD) discovery and benchmark it against the depth-first baseline FDMiner [53] on representative FD datasets Adult and Hospital [54]. As shown in Figure 7(h), transplanting MCTS for FD discovery consistently beats FDMiner, achieving speedups of $3.87\times$ on Adult and $1.39\times$ on Hospital. This improvement arises from MCTS's explicit balance between exploration and exploitation.

Despite exploring only a fraction of paths, PDCRLeaner recovers 89% of the rules found by FDMiner on Adult and 92% on Hospital, showing that MCTS can retrieve near-complete rule sets under limited budgets at substantially lower cost.

Exp-5: Ablation study. We conducted an ablation study.

Without ML policy. We compared PDCRLeaner without the policy model in MCTS (Figure 4(d)). PDCRLeaner struggles to explore the huge search space and fails to identify valuable paths within the iteration budgets. As a result, it discovers fewer valid DCRs, *e.g.,* F1-score drops by 0.16 on Fakeddit.

Without HER. We also tested a variant of PDCRLeaner that discovers DCRs only on relations, without multi-modal HER (Figure 4(d)). The F1-score drops by 0.23 on average, since it misses complementary information from unstructured data for effective ED; this justifies the need of cross-modal cleaning.

Impact of ML accuracy. We evaluated how the accuracy of embedded ML models ($\mathcal{M}_U$, $\mathcal{M}_R$, and $\mathcal{M}_{\text{HER}}$) affects ED on PAD. We manually verified 300 and 100 random samples of predictions from $\mathcal{M}_U$ and $\mathcal{M}_R$, respectively. Their accuracies are 0.76 and 0.74, indicating moderately noise but usable signals. The accuracy of $\mathcal{M}_{\text{HER}}$ is evaluated separately in Exp-3.

To assess robustness, we synthetically corrupted $x\%$ ML predictions (set to empty), and reran RD and ED on PAD. Table III reports the results. While higher ML accuracy

improves ED performance, the impact of noise is limited, *e.g.,* even with 15% corruption on $\mathcal{M}_R$, the F1 drops by only 0.023.

This robustness stems from the design of DCRs: instead of relying on a single ML signal, DCRs synthesize heterogeneous predicates and encode how multiple signals must agree, or how one signal may override another (Section II-B). Thus, spurious ML predictions are less likely to propagate, and DCR-based ED is resilient to isolated ML prediction errors.

Summary. We find the following. (1) *Accuracy*: DCRs learned by PDCRLeaner is effective in ED; our F1 score is 0.61, 0.47 and 0.79 higher than HyperLogic, OHunt and VLM, respectively. (2) *Cross-domain integration*: Incorporating unstructured data improves F1 from 0.38 to 0.80 on Fakeddit, confirming the value of multi-modal cleaning. (3) *Efficiency*: DCR discovery is efficient, *e.g.,* it takes 54.9 min to discover rules from a dataset with 5.01M instances. Its policy model in MCTS improves the F1 of ED by 0.16 on Fakeddit. (4) *HER*: The distillation-based HER beats LLaVA-NeXT and BLIP on most datasets. (5) *Parameter settings*: We find that $\sigma = 10^{-6}|\mathcal{D}|^2$, $\delta = 0.7$, $\alpha = 0.4$ and $c = 1.4$ balance efficiency and accuracy, *e.g.,* it takes 20 min to learn DCRs on Fakeddit with 21.9K instances, while achieving F1 = 0.80 for ED.

VII. Related Work

Dependencies. We classify dependencies by modalities: (1) *Tabular Dependencies*: Defined over relational tables, these include traditional and extended forms such as FDs, CFDs [3], PFDs [55], DCs [2] and REEs [4]. (2) *Graph Dependencies*: These incorporate structural patterns into rules, *e.g.,*GFDs [56], GCRs [57] and GARs [45], extending dependency reasoning to graphs. (3) *Multi-Modal Dependencies*: Frameworks like LNN [58], FOL-LNN, NeuPSL [59] and Δ-FOL [60] combine neural perception and logical reasoning. They support text, images and visual question answering through logic-augmented neural architectures. (4) *RAG-Based*: Retrieval-Augmented Generation (RAG) frameworks, *e.g.,* GraphRAG [61], LightRAG [62], PIKE-RAG [63] and KAG [64], leverage graph rules to build multi-modal knowledge graphs by extracting entities and relations from diverse sources like documents and images.

This work differs from prior works. (1) Most existing methods require specialized neural networks to integrate logical reasoning within neuro-symbolic frameworks, *e.g.,* Δ-FOL relies solely on representation learning to generate visual reasoning with FOL rules, while LNN is based on special RNNs. In contrast, DCRs can unify arbitrary ML models, thus achieving better expressivity and reasoning capabilities. (2) Although some existing methods support multi-modalities, they typically focus on only one or two modalities within a single rule format, *e.g.,* Δ-FOL focuses on text and images. In contrast, DCRs can embed multiple modalities (text, image, audio, video) in a single rule. (3) DCRs do not require unstructured data extraction or data conversion in advance.

Learning for reasoning. Several methods have been studied to learn rules. (1) *Differentiable Methods*: They relax discrete logic into differentiable forms for end-to-end learning, *e.g.,* [65] proposes a noise-resilient differentiable ILP (Inductive Logic Programming) framework; NeurLP [66] and DRUM [67] learn rules for path-based KG completion; NLIL [68] mines subgraph patterns using first-order logic. To improve interpretability, HyperLogic [47] integrates hypernetworks with rule learners to find human-understandable rules. (2) *Non-Differentiable Methods*: They use reinforcement learning (RL) or Expectation-Maximization (EM), *e.g.,* [68] combines image-based input extraction with REINFORCE for few-shot rule learning; RNNLogic [69] and pLogicNet [70] apply EM to co-learn rule structures and neural parameters; NUDGE [71] uses actor-critic RL to learn policies that generate logic rules; OHunt [48] learns logic rules for detecting outliers via meta learning. (3) *Search-Based Methods*: These combine symbolic search with policy learning, *e.g.,* rStar [72] uses Monte Carlo Tree Search (MCTS) guided by LLM-based policies for solving math problems; AgentQ [73] collects trajectories via MCTS and fine-tunes LLM policies using DPO [74].

Unlike prior learning-based reasoning, DCRs unify reasoning across multi-modal data, by combining symbolic logic and statistical prediction in one framework, beyond single modal reasoning like [48] and [69]. The learning of DCRs is the first effort to combine policy learning with rule discovery to optimize MCTS-guided search; this contrasts with existing search methods that use LLM policies solely for symbolic search.

Multi-modal HER. Multi-modal Heterogeneous Entity Resolution (HER) is related to data alignment and fusion. (1) *Data Alignment* ensures coherence across modalities by matching corresponding entities. (a) Explicit alignment uses direct mappings across modalities [75], [76]. (b) Implicit alignment relies on latent representations for cross-modal matching [77], [78]. (2) *Data Fusion* combines information from multiple modalities into a unified representation. (a) Feature-level fusion merges features into a shared space [79]. (b) Model-level fusion integrates models trained on separate modalities [80], [81]. (c) Kernel-based fusion applies kernel methods to blend modality-specific signals [82]. (d) Attention-based fusion uses attention mechanisms to adaptively weight modalities [83].

Our distillation HER uses attention-based data fusion to resolve cross-modal links, aligning structured and unstructured data in a unified space. Unlike prior work, it jointly handles relations, documents, images, video, and audio with modality-agnostic fusion and human-efficiency training strategies.

VIII. Conclusion

The novelty of the work includes (1) DCRs that can detect errors across structured and unstructured data; (2) the complexity bounds of reasoning about DCRs; (3) a distillation-based methods for heterogeneous entity resolution; and (4) an MCTS-guided parallel algorithm for learning DCRs. Our experiments have verified that the learned DCRs can catch errors that cannot be detected by existing data quality rules.

Topics for future work include (a) fixing errors with DCRs in multi-modal data, (b) integrating DCRs with DBMSs, and (c) applying DCRs for improving training data for ML models.

IX. AI-Generated Content Acknowledgment

The content (including but not limited to text, figures, images, and code) was not generated by artificial intelligence.

References

[1] E. F. Codd, "Relational completeness of data base sublanguages," *In: R. Rustin (ed.): Database Systems: 65-98, Prentice Hall and IBM Research Report RJ 987, San Jose, California*, 1972.

[2] M. Arenas, L. Bertossi, and J. Chomicki, "Consistent query answers in inconsistent databases," in *PODS*, 1999, pp. 68–79.

[3] W. Fan, F. Geerts, X. Jia, and A. Kementsietsidis, "Conditional functional dependencies for capturing data inconsistencies," *ACM Trans. Database Syst.*, vol. 33, no. 2, pp. 6:1–6:48, 2008.

[4] W. Fan, C. Tian, Y. Wang, and Q. Yin, "Parallel discrepancy detection and incremental detection," *PVLDB*, vol. 14, no. 8, pp. 1351–1364, 2021.

[5] S. Mishra and A. Misra, "Structured and unstructured big data analytics," in *International Conference on Current Trends in Computer, Electrical, Electronics and Communication (CTCEEC)*. IEEE, 2017, pp. 740–746.

[6] J. Henry, Y. Pylypchuk, T. Searcy, and V. Patel, "Adoption of electronic health record systems among us non-federal acute care hospitals: 2008–2015," *ONC data brief*, vol. 35, no. 35, pp. 2008–15, 2016.

[7] D. Zhang, C. Yin, J. Zeng, X. Yuan, and P. Zhang, "Combining structured and unstructured data for predictive models: A deep learning approach," *BMC medical informatics and decision making*, vol. 20, pp. 1–11, 2020.

[8] R. Koppel and S. Target, "Patient safety and health information technology: Learning from our mistakes," *AHRQ Web M&M. July*, 2012.

[9] S. Yadav, N. Kazanji, N. KC, S. Paudel, J. Falatko, S. Shoichet, M. Maddens, and M. A. Barnes, "Comparison of accuracy of physical examination findings in initial progress notes between paper charts and a newly implemented electronic health record," *Journal of the American Medical Informatics Association*, vol. 24, no. 1, pp. 140–144, 2017.

[10] O. of the National Coordinator for Health Information Technology, "2016 report to congress on health it progress: Examining the hitech era and the future of health it," 2016.

[11] A. G. Pacheco, G. R. Lima, A. S. Salomão, B. Krohling, I. P. Biral, G. G. de Angelo, F. C. Alves Jr, J. G. Esgario, A. C. Simora, P. B. Castro, F. B. Rodrigues, P. H. Frasson, R. A. Krohling, H. Knidel, M. C. Santos, R. B. do Espírito Santo, T. L. Macedo, T. R. Canuto, and L. F. de Barros, "Pad-ufes-20: A skin lesion dataset composed of patient data and clinical images collected from smartphones," *Data in Brief*, 2020.

[12] W. Fan, P. Lu, and C. Tian, "Unifying logic rules and machine learning for entity enhancing," *Sci. China Inf. Sci.*, vol. 63, no. 7, 2020.

[13] X. Bao, Z. Bao, Q. Duan, W. Fan, H. Lei, D. Li, W. Lin, P. Liu, Z. Lv, M. Ouyang, J. Peng, J. Zhang, R. Zhao, S. Tang, S. Zhou, Y. Wang, Q. Wei, M. Xie, J. Zhang, X. Zhang, R. Zhao, and S. Zhou, "Rock: Cleaning data by embedding ml in logic rules," in *SIGMOD (industrial track)*, 2024, pp. 106–119.

[14] Z. Tang, Z. Zhong, T. He, and G. Friedland, "Bag of tricks for multimodal automl with image, text, and tabular data," *CoRR*, vol. abs/2412.16243, 2024.

[15] "Code, datasets and full version," 2024, https://github.com/yyssl88/DCRs/tree/main.

[16] S. Abiteboul, R. Hull, and V. Vianu, *Foundations of Databases*. Addison-Wesley, 1995.

[17] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, W. Che, X. Yu, and F. Wei, "Beats: Audio pre-training with acoustic tokenizers," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 5178–5193.

[18] K. Hu, Z. Wu, Z. Zhong, W. Lin, L. Sun, and Q. Huo, "A question-answering approach to key value pair extraction from form-like document images," in *Conference on Artificial Intelligence (AAAI)*. AAAI Press, 2023, pp. 12 899–12 906.

[19] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, 2019, pp. 4171–4186.

[20] Q. Team, "Qwen2.5-vl," January 2025. [Online]. Available: https://qwenlm.github.io/blog/qwen2.5-vl/

[21] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, Y. Fan, K. Dang, M. Du, X. Ren, R. Men, D. Liu, C. Zhou, J. Zhou, and J. Lin, "Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution," *arXiv preprint arXiv:2409.12191*, 2024.

[22] J. Ooms, *Tesseract: Open Source OCR Engine*, 2025, R package version 5.2.2: https://docs.ropensci.org/tesseract/, https://ropensci.r-universe.dev/tesseract. [Online]. Available: https://docs.ropensci.org/tesseract/https://ropensci.r-universe.dev/tesseract

[23] J. Yoon, J. Jordon, and M. van der Schaar, "GAIN: Missing data imputation using generative adversarial nets," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 5675–5684.

[24] J. Li, D. Li, S. Savarese, and S. C. H. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *International Conference on Machine Learning, ICML*, vol. 202. PMLR, 2023.

[25] B. Lin, Y. Ye, B. Zhu, J. Cui, M. Ning, P. Jin, and L. Yuan, "Video-LLaVA: Learning united visual representation by alignment before projection," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[26] Q. Team, "Qwen3-vl technical report," *CoRR*, vol. abs/2511.21631, 2025.

[27] S. K. Bell, T. Delbanco, J. G. Elmore, P. S. Fitzgerald, A. Fossa, K. Harcourt, S. G. Leveille, T. H. Payne, R. A. Stametz, J. Walker, and C. M. DesRoches, "Frequency and types of patient-reported errors in electronic health record ambulatory care notes," *JAMA network open*, vol. 3, no. 6, pp. e205 867–e205 867, 2020.

[28] W. Fan, H. Gao, X. Jia, J. Li, and S. Ma, "Dynamic constraints for record matching," *VLDB J.*, vol. 20, no. 4, pp. 495–520, 2011.

[29] W. Fan, Z. Han, Y. Wang, and M. Xie, "Parallel rule discovery from large datasets by sampling," in *SIGMOD*. ACM, 2022, pp. 384–398.

[30] M. Garey and D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.

[31] X. Zheng, J. Jia, S. Guo, J. Chen, L. Sun, Y. Xiong, and W. Xu, "Full parameter time complexity (FPTC): A method to evaluate the running time of machine learning classifiers for land use/land cover classification," *IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens.*, vol. 14, pp. 2222–2235, 2021.

[32] B. Shah and H. Bhavsar, "Time complexity in deep learning models," *Procedia Computer Science*, vol. 215, pp. 202–210, 2022.

[33] S. Li and H. Tang, "Multimodal alignment and fusion: A survey," *Int. J. Comput. Vis.*, vol. 134, no. 3, p. 103, 2026.

[34] S. Lin, C. Lee, M. Shoeybi, J. Lin, B. Catanzaro, and W. Ping, "MM-Embed: Universal multimodal retrieval with multimodal LLMs," *CoRR*, vol. abs/2411.02571, 2024.

[35] W. Fan, R. Tugay, Y. Wang, M. Xie, and M. A. Ali, "Learning and deducing temporal orders," *PVLDB*, vol. 16, no. 8, pp. 1944–1957, 2023.

[36] R. G. Praveen and J. Alam, "Recursive joint cross-modal attention for multimodal fusion in dimensional emotion recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 4803–4813.

[37] M. Balcan, A. Z. Broder, and T. Zhang, "Margin based active learning," in *Annual Conference on Learning Theory (COLT)*, ser. Lecture Notes in Computer Science, vol. 4539. Springer, 2007, pp. 35–50.

[38] C. P. Kruskal, L. Rudolph, and M. Snir, "A complexity theory of efficient parallel algorithms," *Theor. Comput. Sci.*, vol. 71, no. 1, pp. 95–132, 1990.

[39] W. Fan, Z. Han, Y. Wang, and M. Xie, "Discovering top-k rules using subjective and objective criteria," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 70:1–70:29, 2023.

[40] W. Fan, Z. Han, M. Xie, and G. Zhang, "Discovering top-k relevant and diversified rules," *Proc. ACM Manag. Data*, vol. 2, no. 4, pp. 195:1–195:28, 2024.

[41] E. H. Pena, E. C. De Almeida, and F. Naumann, "Discovery of approximate (and exact) denial constraints," *VLDB*, 2019.

[42] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. P. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[43] G. Bosc, J.-F. Boulicaut, C. Raïssi, and M. Kaytoue, "Anytime discovery of a diverse set of patterns with monte carlo tree search," *Data mining and knowledge discovery*, vol. 32, pp. 604–650, 2018.

[44] P. Ding, G. Liu, Y. Wang, K. Zheng, and X. Zhou, "A-MCTS: Adaptive Monte Carlo tree search for temporal path discovery," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 3, pp. 2243–2257, 2021.

[45] W. Fan, W. Fu, R. Jin, P. Lu, and C. Tian, "Discovering association rules from big graphs," *PVLDB*, vol. 15, no. 7, pp. 1479–1492, 2022.

[46] L. G. Valiant, "A bridging model for parallel computation," *Commun. ACM*, vol. 33, no. 8, pp. 103–111, 1990.

[47] Y. Yang, W. Ren, and S. Li, "Hyperlogic: Enhancing diversity and accuracy in rule learning with hypernets," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

[48] S. Chen, W. Fan, and R. Jin, "Outliers: The good, the bad and the ugly," *Proc. ACM Manag. Data*, vol. 3, no. 4, pp. 259:1–259:30, 2025.

[49] F. Li, R. Zhang, H. Zhang, Y. Zhang, B. Li, W. Li, Z. Ma, and C. Li, "Llava-next-interleave: Tackling multi-image, video, and 3d in large multimodal models," *arXiv preprint arXiv:2407.07895*, 2024.

[50] D. Li, J. Li, H. Le, G. Wang, S. Savarese, and S. C. H. Hoi, "Lavis: A library for language-vision intelligence," 2022.

[51] Octaprice team, "Ecommerce product dataset," 2025, *https://github.com/octaprice/ecommerce-product-dataset*.

[52] S. Aptlin, "Posterlens 25m." 2021, doi: 10.34740/KAGGLE/DS/1321802.

[53] C. Wyss, C. Giannella, and E. Robertson, "FastFDs: A heuristic-driven, depth-first algorithm for mining functional dependencies from relation instances extended abstract," in *DaWaK*, 2001.

[54] X. Chu, I. F. Ilyas, and P. Papotti, "Discovering denial constraints," *PVLDB*, 2013.

[55] A. Qahtan, N. Tang, M. Ouzzani, Y. Cao, and M. Stonebraker, "Pattern functional dependencies for data cleaning," *PVLDB*, vol. 13, no. 5, pp. 684–697, 2020.

[56] A. Cortés-Calabuig and J. Paredaens, "Semantics of constraints in RDFS," in *AMW*, 2012.

[57] W. Fan, W. Fu, R. Jin, P. Lu, and C. Tian, "Making it tractable to catch duplicates and conflicts in graphs," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 86:1–86:28, 2023.

[58] R. Riegel, A. G. Gray, F. P. S. Luus, N. Khan, N. Makondo, I. Y. Akhalwaya, H. Qian, R. Fagin, F. Barahona, U. Sharma, S. Ikbal, H. Karanam, S. Neelam, A. Likhyani, and S. K. Srivastava, "Logical neural networks," *CoRR*, vol. abs/2006.13155, 2020.

[59] C. Pryor, C. Dickens, E. Augustine, A. Albalak, W. Y. Wang, and L. Getoor, "NeuPSL: Neural probabilistic soft logic," in *International Joint Conference on Artificial Intelligence (IJCAI)*. ijcai.org, 2023, pp. 4145–4153.

[60] S. Amizadeh, H. Palangi, A. Polozov, Y. Huang, and K. Koishida, "Neuro-symbolic visual reasoning: Disentangling "visual" from "reasoning"," in *International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 279–290.

[61] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *CoRR*, vol. abs/2404.16130, 2024.

[62] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "Lightrag: Simple and fast retrieval-augmented generation," *CoRR*, vol. abs/2410.05779, 2024.

[63] J. Wang, J. Fu, R. Wang, L. Song, and J. Bian, "PIKE-RAG: sPecIalized KnowledgE and Rationale Augmented Generation," *CoRR*, vol. abs/2501.11551, 2025.

[64] L. Liang, M. Sun, Z. Gui, Z. Zhu, Z. Jiang, L. Zhong, Y. Qu, P. Zhao, Z. Bo, J. Yang, H. Xiong, L. Yuan, J. Xu, Z. Wang, Z. Zhang, W. Zhang, H. Chen, W. Chen, and J. Zhou, "KAG: Boosting LLMs in professional domains via knowledge augmented generation," *CoRR*, vol. abs/2409.13731, 2024.

[65] R. Evans and E. Grefenstette, "Learning Explanatory Rules from Noisy Data," *Journal of Artificial Intelligence Research (JAIR)*, vol. 61, pp. 1–64, 2018. [Online]. Available: https://jair.org/index.php/jair/article/view/11172

[66] F. Yang, Z. Yang, and W. W. Cohen, "Differentiable learning of logical rules for knowledge base reasoning," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2017, pp. 2319–2328.

[67] A. Sadeghian, M. Armandpour, P. Ding, and D. Z. Wang, "DRUM: End-to-end differentiable rule mining on knowledge graphs," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2019.

[68] Y. Yang and L. Song, "Learn to explain efficiently via neural logic inductive learning," in *International Conference on Learning Representations (ICLR)*, Apr. 2020.

[69] M. Qu, J. Chen, L. A. C. Xhonneux, Y. Bengio, and J. Tang, "RNN-Logic: Learning logic rules for reasoning on knowledge graphs," in *International Conference on Learning Representations (ICLR)*, 2021.

[70] M. Qu and J. Tang, "Probabilistic logic neural networks for reasoning," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2019.

[71] Q. Delfosse, H. Shindo, D. S. Dhami, and K. Kersting, "Interpretable and explainable logical policies via neurally guided symbolic abstraction," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2023.

[72] X. Guan, L. L. Zhang, Y. Liu, N. Shang, Y. Sun, Y. Zhu, F. Yang, and M. Yang, "rStar-Math: Small LLMs can master math reasoning with self-evolved deep thinking," *CoRR*, vol. abs/2501.04519, 2025.

[73] P. Putta, E. Mills, N. Garg, S. Motwani, C. Finn, D. Garg, and R. Rafailov, "Agent Q: Advanced reasoning and learning for autonomous AI agents," *CoRR*, vol. abs/2408.07199, 2024.

[74] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *CoRR*, vol. abs/2305.18290, 2023.

[75] G. Andrew, R. Arora, J. A. Bilmes, and K. Livescu, "Deep canonical correlation analysis," in *International Conference on Machine Learning (ICML)*, ser. JMLR Workshop and Conference Proceedings, vol. 28. JMLR.org, 2013, pp. 1247–1255.

[76] Y. Verma and C. V. Jawahar, "Im2Text and Text2Im: Associating images and texts for cross-modal retrieval," in *British Machine Vision Conference (BMVC)*. BMVA Press, 2014.

[77] M. Tao, B. Bao, H. Tang, and C. Xu, "GALIP: Generative adversarial clips for text-to-image synthesis," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, (CVPR)*. IEEE, 2023, pp. 14 214–14 223.

[78] H. Tang and N. Sebe, "Facial expression translation using landmark guided gans," 2022. [Online]. Available: https://arxiv.org/abs/2209.02136

[79] F. Scalzo, G. Bebis, M. Nicolescu, L. A. Loss, and A. Tavakkoli, "Feature fusion hierarchies for gender classification," in *International Conference on Pattern Recognition (ICPR)*. IEEE Computer Society, 2008, pp. 1–4.

[80] C. Guo and L. Zhang, "A model-level fusion-based multi-modal object detection and recognition method," in *Asian Conference on Artificial Intelligence Technology (ACAIT)*. IEEE, 2023, pp. 34–38.

[81] Y. Wei, D. Wu, and J. Terpenny, "Decision-level data fusion in quality control and predictive maintenance," *IEEE Transactions on Automation Science and Engineering*, vol. 18, no. 1, pp. 184–194, 2020.

[82] Y. Wang, L. Guan, and A. N. Venetsanopoulos, "Kernel cross-modal factor analysis for information fusion with application to bimodal emotion recognition," *IEEE Trans. Multim.*, vol. 14, no. 3-1, pp. 597–607, 2012.

[83] J. Yu, Z. Wang, V. Vasudevan, L. Yeung, M. Seyedhosseini, and Y. Wu, "Coca: Contrastive captioners are image-text foundation models," *Trans. Mach. Learn. Res.*, vol. 2022, 2022.

[84] M. Li, Y. Fu, and Y. Zhang, "Spatial-spectral transformer for hyperspectral image denoising," in *AAAI*. AAAI Press, 2023, pp. 1368–1376.

[85] Y. Yao, T. Ghai, S. Ravi, and P. A. Szekely, "AMPPERE: A universal abstract machine for privacy-preserving entity resolution evaluation," in *CIKM*, 2021, pp. 2394–2403.

[86] C. C. Aggarwal, Ed., *Data Classification: Algorithms and Applications*. CRC Press, 2014.

[87] A. C. Klug, "On conjunctive queries containing inequalities," *J. ACM*, vol. 35, no. 1, pp. 146–160, 1988.

[88] M. Baudinet, J. Chomicki, and P. Wolper, "Constraint-generating dependencies," *JCSS*, 1999.

APPENDIX

A1. PROOF OF THEOREM 1

We show that satisfiability, implication and validation are NP-complete, Π_2^p-complete and coNP-complete, respectively.

Satisfiability. We start with the upper bound.

Upper bound. We construct several canonical datasets based on the relation schemas and the unstructured data in Σ. Then we convert the set Σ into multiple sets of Horn formulas, one for each canonical dataset. Finally, we present an NP algorithm for satisfiability using these sets of Horn formulas.

Canonical datasets. Let $R_1, \ldots, R_n$ be all relation schemas within the set Σ, and $U_1, \ldots, U_m$ be all types of unstructured data in Σ. We define $n+m$ canonical datasets $(\mathcal{D}_i, \mathcal{U}_i)$ $(i \in [1, n+m])$ as follows: (a) for $i \in [1,n]$, $\mathcal{D}_i$ consists of only one tuple t_i in relation schema R_i, with a variable x_j^i for each attribute A_j^i in R_i. All other relations and unstructured data are empty. (b) For $i \in [n+1, n+m]$, the unstructured data $\mathcal{U}_i$ consists of only one object o_i, yet another distinct variable. Each object o_i carries a variable y_j^i for each attribute A_j^i in Σ. All relations and other unstructured datasets are empty.

Intuitively, each canonical dataset $(\mathcal{D}_i, \mathcal{U}_i)$ consists of only one tuple or object. If there is some canonical dataset $(\mathcal{D}_i, \mathcal{U}_i)$ such that $(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$, then the set Σ is satisfiable.

Horn formulas. We construct a set of Horn formulas from Σ and a canonical dataset. Here a Horn formula is $W_1 \wedge \ldots \wedge W_k \rightarrow V$, where $W_1, \ldots W_k$ and V are Boolean variables.

Given Σ and $(\mathcal{D}_i, \mathcal{U}_i)$, we define a set Σ_H^i of DCRs:

(1) For tuple t_i in relation schema R_i, if variable x_j^i appears in attribute A_j^i, Σ_H^i includes Horn formula $\emptyset \rightarrow W_{t_i.A_j^i = x_j^i}$, where $W_{t_i.A_j^i = x_j^i}$ is a Boolean variable denoting $t_i.A_j^i = x_j^i$.

(2) For object o_i in $\mathcal{U}_i$, Σ_H^i includes formula $\emptyset \rightarrow W_{o_i.A_j^i = y_j^i}$, where $W_{o_i.A_j^i = y_j}$ indicates that $o_i.A_j^i = y_j^i$.

(3) For each DCR $\varphi = p_1 \wedge \ldots \wedge p_n \rightarrow p_0$ in Σ, the set Σ_H^i includes one Horn formula $W_{h(p_1)} \wedge \ldots \wedge W_{h(p_n)} \rightarrow W_{h(p_0)}$, where h is the unique valuation of φ in $(\mathcal{D}_i, \mathcal{U}_i)$. Recall that $(\mathcal{D}_i, \mathcal{U}_i)$ consists of only one tuple or object. Here $h(p_i)$ is a predicate deduced from p_i by replacing tuple variable t (resp. external variable u and object variable o) by $h(t)=t_i^j$ (resp. $h(u)=e_i$ and $h(o)=o_i$), and $W_{h(p_i)}$ denotes predicate $h(p_i)$.

(4) To avoid assigning distinct constants to the same attribute of a tuple or an object, we add two Horn formulas: $Z_{t_i.A_i^j = c} \wedge Z_{t_i.A_i^j = d} \rightarrow \mathsf{false}$ and $Z_{o_i.A_i^j = c} \wedge Z_{o_i.A_i^j = d} \rightarrow \mathsf{false}$ for any two distinct constants c and d appearing in Σ.

We show the following two properties.

(1) The set Σ is satisfiable if and only if (a) Σ_H^i is satisfiable for some $i \in [1, n+m]$, and (b) there exists a satisfying truth assignment for Σ_H^i that has an instance $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ from the canonical dataset $(\mathcal{D}_i, \mathcal{U}_i)$, such that $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$. Here $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ is the mixed dataset created by substituting variables in $(\mathcal{D}_i, \mathcal{U}_i)$ with constants.

(2) Given a satisfying assignment μ of Σ_H^i, it is in PTIME to check if μ deduces $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ satisfying $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$.

Proof of Property (1). When Σ_H^i has some satisfying truth assignment that deduces an instance $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ such that $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$, obviously Σ is satisfiable. Here we primarily prove the other direction.

Suppose that Σ is satisfiable. Then there exists a mixed dataset $(\mathcal{D}, \mathcal{U})$ such that $(\mathcal{D}, \mathcal{U}) \models \Sigma$. We can deduce a satisfying truth assignment for Σ_H^i for some $i \in [1, n+m]$, leading to an instance $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ that satisfies $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$. To this end, we first identify an instance $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ of some canonical dataset $(\mathcal{D}_i, \mathcal{U}_i)$, from which we deduce a satisfying truth assignment. We start with the construction of $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$. (1) When $\mathcal{D} \neq \emptyset$, let t be a tuple in some relation D_i of relation schema R_i. Then we define a mixed dataset $(D_i^t, \emptyset)$, where D_i^t consists of only tuple t. Obviously, $(D_i^t, \emptyset)$ is an instance of canonical dataset $(\mathcal{D}_i, \mathcal{U}_i)$. (2) When $\mathcal{U} \neq \emptyset$, let o_i be an object in some unstructured data $\mathcal{U}_i$. We define a mixed dataset $(\emptyset, U_i^o)$, where U_i^o consists of only one object o_i. Similarly, $(\emptyset, U_i^o)$ is an instance of some canonical dataset $(\mathcal{D}_i, \mathcal{U}_i)$. For simplicity, denote by (D_i^t, U_i^o) the constructed dataset. Let Σ_H^i be the set of Horn formulas that are derived from the canonical dataset $(\mathcal{D}_i, \mathcal{U}_i)$.

We next deduce a truth assignment μ for Σ_H^i using (D_i^t, U_i^o). (a) For each variable $W_{t_i.A_j^i = x_j^i}$ that is defined on relation D_i consisting of the unique tuple t, we define $\mu(W_{t.A_j^i = x_j^i}) = \mathsf{true}$, where x_j^i is the value of attribute A_j^i in tuple t. (b) For Boolean variable $W_{o_i.A_j^i = y_j^i}$ that is defined on the unstructured dataset $\mathcal{U}_i$, consisting of the unique object o_i, we set $\mu(W_{o_i.A_j^i = y_j^i}) = \mathsf{true}$. (3) For each DCR $\varphi = p_1 \wedge \ldots \wedge p_n \rightarrow p_0$ in Σ, given that (D_i^t, U_i^o) consists of only one tuple or object, there exists exactly one valuation h from φ in (D_i^t, U_i^o). For each variable $W_{h(p_l)}$, if $h(p_l)$ is true in (D_i^t, U_i^o), then $W_{h(p_l)}$ is assigned true, and false otherwise. (4) For Boolean variables in Horn formulas of the form $Z_{t_i.A_i^j = c} \wedge Z_{t_i.A_i^j = d} \rightarrow \mathsf{false}$ and $Z_{o_i.A_i^j = c} \wedge Z_{o_i.A_i^j = d} \rightarrow \mathsf{false}$, if $t_i.A_i^j = c$ (resp. $o_i.A_i^j = c$) holds in (D_i^t, U_i^o), then we define $\mu(Z_{t_i.A_i^j = c}) = \mathsf{true}$ and $\mu(Z_{o_i.A_i^j = c}) = \mathsf{true}$; similarly when $t_i.A_i^j = d$ and $o_i.A_i^j = d$ hold in (D_i^t, U_i^o).

We next show that the deduced truth assignment μ satisfies all DCRs in Σ_H^i. (a) For each DCRs in the forms of $\emptyset \rightarrow W_{t_i.A_j^i = x_j^i}$, since $\mu(W_{t_i.A_j^i = x_j^i}) = \mathsf{true}$, it follows that μ satisfies these Horn formulas; similarly for DCRs in the form of $\emptyset \rightarrow W_{o_i.A_j^i = y_j^i}$. (b) For DCRs in the forms of $Z_{t_i.A_i^j = c} \wedge Z_{t_i.A_i^j = c} \rightarrow \mathsf{false}$ and $Z_{o_i.A_i^j = c} \wedge Z_{o_i.A_j^i = d} \rightarrow \mathsf{false}$, given that (D_i^t, U_i^o) is consistent, *i.e.,* each attribute is assigned only one constant, μ satisfies these Horn formulas. (c) For DCRs in the forms of $\varphi = p_1 \wedge \ldots \wedge p_n \rightarrow p_0$, as $(D_i^t, U_i^o) \models \Sigma$, the truth assignment μ satisfies the Horn formula $W_{h(p_1)} \wedge \ldots \wedge W_{h(p_n)} \rightarrow W_{h(p_0)}$.

Proof of Property (2). That is to develop a PTIME algorithm to check whether a satisfying truth assignment μ of Σ_H^i deduces an instance $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ such that $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$.

Here we assume *w.l.o.g.* that the domain of variables is

dense. Indeed, (1) for attributes $x.A$ in logic predicates $x.A \oplus c$ and $x.A \oplus y.B$, if the ranges of these attributes are $[a, b]$ with $a < b$, then there exist infinitely many real numbers within $[a, b]$, *i.e.,* there must exist a satisfying assignment for attribute $x.A$. (2) For ML predicate $o = \mathcal{M}_U(\mathcal{U}, t)$, there exist infinitely many collections of unstructured data $\mathcal{U}'$ and tuple t' such that $o = \mathcal{M}_U(\mathcal{U}', t')$, *e.g.,* $\mathcal{M}_U$ extracts (a) the same object o carrying infinitely many attributes and values that do not exist in tuple t, (b) extract the same image with different noise levels [84], or (c) the same entity in privacy-preserving settings [85]; similarly for HER operators $\Theta_{\asymp}(t, o)$ and ML predicates $\mathcal{M}_R(e_1, e_2)$. Such assumptions are guaranteed by most high dimensional ML predicates [86], [12], [84], [85].

To present the algorithm, we partition variables in Σ_H^i into two sets $L^{\oplus}$ and $L^{\mathcal{M}}$, where $L^{\oplus}$ consists of Boolean variables in the form of $W_{e.A \oplus c}$ and $W_{e_1.A \oplus e_2.B}$, and $L^{\mathcal{M}}$ consists of Boolean variables constructed from ML predicates. Intuitively, predicates in $L^{\oplus}$ specify the range of each attribute of tuples or objects in $(\mathcal{D}_i, \mathcal{U}_i)$, from which we can determine whether predicates in $L^{\mathcal{M}}$ are satisfiable. Given a satisfying truth assignment μ of Σ_H^i, we can check whether there exists $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ satisfying $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$ as follows.

(1) Compute the ranges ψ of variables in $(\mathcal{D}_i, \mathcal{U}_i)$ using truth assignments of the Boolean variables in $L^{\oplus}$ within μ.

(2) Check whether there exists a truth assignment for ML predicates in $L^{\mathcal{M}}$ using the ranges ψ; if so, return true.

The correctness follows from definitions of $L^{\oplus}$ and $L^{\mathcal{M}}$.

For the complexity, step (1) is in PTIME, by using the linear order of numbers [87]. Step (2) can be done in PTIME as follows: (a) when all variables in an ML predicate in $L^{\mathcal{M}}$ (*i.e.,* $\mathcal{M}_U(\mathcal{U}, t)$, $\Theta_{\asymp}(t, o)$ or $\mathcal{M}_R(e_1, e_2)$) are instanced with constants (*e.g.,* the range for a variable is a singleton set), the results of the ML predicate are determined; (b) when the range for some variable in an ML predicate is not consistent, *e.g.,* both $x.A \geq 1$ and $x.A < 1$ can be deduced from ψ, the ML predicate is not satisfiable; otherwise, (c) the ML predicate is satisfiable. This is because we assume that the domain of variables is dense, and the ML predicates can output infinitely many true and false, when the input is not fixed.

Algorithm. We provide an NP algorithm to check whether Σ is satisfiable, *i.e.,* there exists $(\mathcal{D}, \mathcal{U})$ such that $(\mathcal{D}, \mathcal{U}) \models \Sigma$.

(1) Construct the set Σ_H^i of formulas for each $i \in [1, n+m]$.

(2) Guess Σ_H^i with $i \in [1, n+m]$ and an assignment μ of Σ_H^i.

(3) Check whether μ is a satisfying truth assignment for Σ_H^i; if so, continue; otherwise, reject the current guess.

(4) Check whether μ deduces an instance $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i)$ satisfying $\mathcal{I}(\mathcal{D}_i, \mathcal{U}_i) \models \Sigma$; if so, return true.

The correctness follows from property (1). The algorithm is in NP, since steps (1) and (3) are in PTIME, and each relation (resp. unstructured data) instance consists of at most one tuple (resp. object). Step (4) is in PTIME due to property (2).

Lower bound. We prove the NP-hardness by reduction from the 3-colorability problem, which is known to be NP-complete [30]. The 3-colorability problem is to decide, given an undirected graph $G = (V, E)$, whether there exists a 3-coloring λ of G such that for each edge (u, v) in G, $\lambda(u) \neq \lambda(v)$. Assume that G consists of n vertices, *i.e.,* $n = |V|$. We construct two sets Σ_D and Σ_U of DCRs for relational data and unstructured data, respectively, such that G is 3-colorable if and only if $\Sigma_D \cup \Sigma_U$ is satisfiable.

We start with the construction of Σ_U, which consists of the following three subsets of DCRs:

(1) The first set Σ_U^1 of DCRs ensures that if $\mathcal{U}$ is not empty, then $\mathcal{U}$ has at least one object o_t, which has n attributes, one for each vertex in G. More specifically, Σ_U^1 consists of n DCRs: $o_t = \mathcal{M}_R(\mathcal{U}) \rightarrow o_t.A_i = o_t.A_i$ for each $i \in [1, n]$; the DCR ensures that the object o_t carries attribute A_i, to encode the color of vertex v_i in G. Recall that $\mathcal{M}_U$ optionally takes a schema R as input, and the DCR ensures that the object o_t carries attribute A_i, to represent the color of vertex v_i in G;

(2) The second set Σ_U^2 of DCRs ensures that the color of each vertex is one of r, g or b: $o_t{=}\mathcal{M}_U(\mathcal{U}) \wedge o_t.A_i{\neq}r \wedge o_t.A_i{\neq}g \wedge o_t.A_i{\neq}b{\rightarrow}$false for each $i \in [1, n]$; the DCR ensures that attribute A_i of object o_t must be one of r, g or b; and

(3) The third set Σ_U^3 of DCRs is to ensure that the coloring is valid. More specifically, for each edge (v_i, v_j) in G, Σ_U^3 contains one DCR: $o_t = \mathcal{M}_U(\mathcal{U}) \wedge o_t.A_i = o_t.A_j \rightarrow$ false, which states that attributes A_i and A_j of object o_t cannot be the same, *i.e.,* the colors of vertices v_i and v_j cannot be the same. Recall that attributes A_i and A_j represent the colors of vertices v_i and v_j in G, respectively.

The construction of Σ_D is similar. Intuitively, DCRs in Σ_D ensure that (i) $\mathcal{D}$ consists of a tuple t with n attributes $A_1, \ldots, A_n$, in which A_i encodes vertex v_i in G; (ii) the values of these attributes are one of r, g or b; and (iii) for each edge (v_i, v_j) in G, $t.A_i{\neq}t.A_j$ holds for tuple t in $\mathcal{D}$ [88].

We define the set Σ of DCRs as $\Sigma_D \cup \Sigma_U$. We can readily show that the construction of Σ is in PTIME.

We prove that G is 3-colorable if and only if Σ is satisfiable.

($\Rightarrow$) When G is 3-colorable, let λ be a valid coloring. We can construct $\mathcal{D}$ satisfying Σ as follows: (a) $\mathcal{D}$ consists of only one tuple t carrying attributes $A_1, \ldots, A_n$; and (b) attribute $t.A_i$ is defined as $\lambda(v_i)$. We can readily verify that $(\mathcal{D}, \emptyset) \models \Sigma$.

($\Leftarrow$) When Σ is satisfiable, there exists a mixed dataset $(\mathcal{D}, \mathcal{U})$ such that $(\mathcal{D}, \mathcal{U}) \models \Sigma$. When $\mathcal{U}$ is not empty, from DCR in Σ_U^1 we know that there exists an object o_t that can be extracted from $\mathcal{U}$ via $\mathcal{M}_U$, and the object o_t carries attributes $A_1, \ldots, A_n$. Then we define a coloring λ of G as follows: $\lambda(v_i) = o_t.A_i$. We can verify that λ is a valid coloring, based on the DCRs in $\Sigma_U^2 \cup \Sigma_U^3$. When $\mathcal{D}$ is not empty, we can similarly prove that G is 3-colorable using DCRs in Σ_D [88].

Implication. The lower bound follows from REEs [12], since REEs are a special case of DCRs, and implication for REEs is Π_2^p-complete. The reduction from the implication problem for REEs to the implication problem for DCRs is given as follows. Given a set Σ of REEs and an REE φ, since REEs

are solely defined on relational datasets, $\Sigma \not\models \varphi$ if and only if there exists a relational dataset $\mathcal{D}$ such that $\mathcal{D} \models \Sigma$ but $\mathcal{D} \not\models \varphi$ if and only if there exists a mixed dataset $(\mathcal{D},\mathcal{U})$ such that $(\mathcal{D},\mathcal{U}) \models \Sigma$ but $(\mathcal{D},\mathcal{U}) \not\models \varphi$.

We show that the implication problem for DCRs is in Π_2^p. To this end, we also construct a canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$, compute a set Σ_H^φ of Horn formulas from the canonical dataset, and check the implication by guessing a satisfying truth assignment for these Horn formulas. Different from the proof for the satisfiability problem, (1) we construct only one canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ using only relation schemas and unstructured data constrained in the DCR φ; and a relation may consist of multiple tuples; (2) for the given DCR $\varphi = p_1 \wedge \ldots \wedge p_n \rightarrow p_0$, we add the following two types of Horn formulas: (a) $\emptyset \rightarrow W_{p_i}$ for each p_i in the precondition of φ; and (b) $W_{p_0} \rightarrow \mathsf{false}$ for the consequence p_0 of φ; and (3) for each DCR $\varphi' = p'_1 \wedge \ldots \wedge p'_n \rightarrow p_0$ in Σ, Σ_H^i includes one Horn formula $W_{h(p'_1)} \wedge \ldots \wedge W_{h(p'_n)} \rightarrow W_{h(p'_0)}$ for each valuation h of φ in the unique canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$, as in the proof for satisfiability. There may exist exponentially many formulas in Σ_H^φ, since each relation contains multiple tuples in $\mathcal{D}$, and there may exist exponentially many valuations from a DCR $\varphi' \in \Sigma$ in $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$.

Given a truth assignment μ for variables in Σ_H^φ, we do not explicitly construct Σ_H^φ, to check whether Σ_H^φ is satisfied by μ. Instead, we develop an NP algorithm to check whether Σ_H^φ *is not satisfied* by μ as follows: (1) guess a DCR $\varphi' = p'_1 \wedge \ldots \wedge p'_n \rightarrow p_0$ in Σ and a valuation h' from φ' in the canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$; (2) construct the Horn formula $W_{h'(p'_1)} \wedge \ldots \wedge W_{h'(p'_n)} \rightarrow W_{h'(p'_0)}$ using h' as discussed above; and (3) check whether the Horn formula *is not satisfied* by μ; if so, return true, and false otherwise.

We can verify that (a) $\Sigma \not\models \varphi$ if and only if Σ_H^φ is satisfiable and there exists a satisfying truth assignment μ of Σ_H^φ that deduces an instance $\mathcal{I}(\mathcal{D}_i,\mathcal{U}_i)$ such that $\mathcal{I}(\mathcal{D}_i,\mathcal{U}_i) \models \Sigma$ but $\mathcal{I}(\mathcal{D}_i,\mathcal{U}_i) \not\models \varphi$; (b) given a set Σ of DCRs, there exist polynomial many variables in Σ_H^φ; and (c) it is in PTIME to check if a satisfying truth assignment μ of Σ_H^φ deduces such an instance $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$.

Proof of Property (a). Assume that Σ_H^φ is satisfied by some truth assignment μ of Σ_H^φ. If the truth assignment μ can deduce an instance $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ such that $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \models \Sigma$ but $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \not\models \varphi$, then obviously $\Sigma \not\models \varphi$. We next prove the other direction.

Assume that $\Sigma \not\models \varphi$. There exists a mixed dataset $(\mathcal{D},\mathcal{U})$ such that $(\mathcal{D},\mathcal{U}) \models \Sigma$ but $(\mathcal{D},\mathcal{U}) \not\models \varphi$. We next derive a truth assignment μ for Σ_H^φ, which deduces an instance $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ of the canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ such that $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \models \Sigma$ but $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \not\models \varphi$. To this end, we first construct the instance $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ from $(\mathcal{D},\mathcal{U})$. Since $(\mathcal{D},\mathcal{U}) \not\models \varphi$, there exists a valuation h from φ in $(\mathcal{D},\mathcal{U})$ such that $h \not\models \varphi$. The instance $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ consists of all tuples $h(t)$, elements $h(u)$ and objects $h(o)$ for tuple variables t, external variables u and object variables o in φ, respectively. One can readily verify that (a) $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ is an instance of the canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$; and (b) $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \models \Sigma$ but $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \not\models \varphi$.

To construct a satisfying truth assignment μ for Σ_H^φ, we utilize valuations from DCRs of Σ in $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$, as in the proof of satisfiability. Unlike the satisfiability problem, for the given DCR $\varphi = p_1 \wedge \ldots \wedge p_n \rightarrow p_0$, we define $\mu(W_{p_i}) = \mathsf{true}$ for each p_i in the precondition of φ and $\mu(W_{p_0}) = \mathsf{false}$. Since $(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \models \Sigma$ and $h \not\models \varphi$, we can show that μ is a satisfying truth assignment for Σ_H^φ.

Proof of Property (b). We show that there exist polynomial many variables in Σ_H^φ. Observe that all variables are in the form of $W_{R(t)}, W_{o=\mathcal{M}_U(\mathcal{U},t)}, W_{\Theta_\asymp(t,o)}, W_{\mathcal{M}_R(e_1,e_2)}, W_{e.A \oplus c}$, $W_{e_1.A \oplus e_1.B}$. Since there are at most $|\mathcal{D}_\varphi|$ and $|\mathcal{U}_\varphi|$ attributes in $\mathcal{D}_\varphi$ and $\mathcal{U}_\varphi$, respectively, there are at most $|\mathcal{D}_\varphi|^2 \times |\mathcal{U}_\varphi|^2$ predicates, *i.e.,* $|\mathcal{D}_\varphi|^2 \times |\mathcal{U}_\varphi|^2$ many variables in Σ_H^φ.

Proof of Property (c). We develop a PTIME algorithm to check whether a truth assignment μ of Σ_H^φ deduces an instance $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ of the canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ such that $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \models \Sigma$ but $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \not\models \varphi$. Given μ, we check the existence of $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ as follows: (1) compute the ranges ψ of variables in the canonical dataset $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ using the truth assignment μ; and (2) check whether there exists a truth assignment for ML predicates in $L^{\mathcal{M}}$ based on ψ. The algorithm is in PTIME, since there exist only polynomial many predicates in Σ_H^φ. The correctness follows from the assumption that the domain of variables is dense.

Algorithm. We give an Σ_2^p algorithm to check whether $\Sigma \not\models \varphi$.

(1) construct the set V^φ of all variables using $(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$;
(2) guess a truth assignment μ for V^φ;
(3) check whether μ satisfies all formulas in Σ_H^φ by guessing a Horn formula in Σ_H^φ that is not satisfied by μ, *i.e.,* an NP algorithm; if not, reject the guess; otherwise, continue;
(4) check whether μ deduces an instance $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi)$ satisfying $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \models \Sigma$ but $\mathcal{I}(\mathcal{D}_\varphi,\mathcal{U}_\varphi) \not\models \varphi$.

The correctness of the algorithm follows from the definition of Σ_H^φ and property (a). The algorithm is in Σ_2^p, since step (1) is in PTIME, due to property (b); step (3) is in NP by guessing a Horn formula in Σ_H^φ and checking whether it is not satisfied by μ; and step (4) is in PTIME due to property (c).

Validation. The lower bound follows from REEs [12]. For the upper bound, given $(\mathcal{D},\mathcal{U})$ and a set Σ of DCRs, we provide a coNP algorithm to check whether $(\mathcal{D},\mathcal{U}) \not\models \Sigma$ or not:

(1) guess a DCR φ and a valuation h from φ in $(\mathcal{D},\mathcal{R})$; and
(2) check whether $h \not\models \Sigma$; if so, return true.

The algorithm is correct by problem statement. It is in coNP since step (2) is in PTIME, assuming that ML models take PTIME to apply, as commonly found in practice [31], [32]. □

A2. Parallel Discovery

Below we first outline the parallel algorithm PDCRLeaner, and then prove its parallel scalability (*i.e.,* Theorem 2).

Algorithm. As shown in Figure 8, given a mixed dataset $(\mathcal{D}, \mathcal{U})$, a requirement req and a maximum number I_{max} of MCTS

Input: $(\mathcal{D}, \mathcal{U})$, $\mathsf{req}(\mathcal{P}_X, \mathcal{P}_0, \sigma, \delta, \eta)$, $f(\cdot)$ and a bound I_{max}.
Output: A cover Σ^c of DCRs conforming to $\mathsf{req}(\mathcal{P}_X, \mathcal{P}_0, \sigma, \delta, \eta)$.
1. Pre-compute ML predictions and train models $\mathcal{M}_p$ and $\mathcal{M}_v$;
2. $\Sigma := \emptyset$; $W := \emptyset$;
3. **for** each p_0 in $\mathcal{P}_0$ **do**
4. $\quad w_0 :=$ a work unit created for p_0; $W := W \cup \{w_0\}$;
5. Evenly distribute work units W to all workers;
6. $\Sigma_i = \mathsf{DCRLeaner}(W_i)$; /* local rule discovery at worker P_i*/
7. **return** $\mathsf{getCover}(\cup_{i=1}^{n} \Sigma_i)$;

Fig. 8: Parallel discovery algorithm PDCRLeaner

iterations, PDCRLeaner returns a set Σ^c of DCRs.

Algorithm PDCRLeaner first pre-computes the ML predictions and trains the policy model and the value model along the same line as sequential discovery (line 1). It then creates a work unit for each consequence p_0 in $\mathcal{P}_0$. The coordinator collects the work units in a set W (lines 2-4), and distributes W evenly to all the workers (line 5). Upon receiving the assigned work units W_j, each worker P_j invokes the sequential discovery algorithm DCRLeaner in parallel and returns a set Σ_i of DCRs for all $p_0 \in W_j$ (line 6). Finally, the algorithm terminates when all the work units have been processed, and returns a cover set Σ^c of all DCRs in all Σ_i's (line 7).

Remark. On real-life datasets, there are many more p_0 (*i.e.,* work units) than the number n of workers. Hence we can balance the workload by evenly distributing them to n workers.

Proof of Theorem 2. We verify Theorem 2 along the same line as [29], [39]. Recall that the complexity of DCRLeaner is $t(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}) = O(c_{\mathsf{valid}} I_{\mathsf{max}} F |P_X| \times |\mathcal{P}_0|)$. We show that the runtime of PDCRLeaner is $O(\frac{t(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req})}{n})$ with n processors.

Let $\mathcal{P}_0 = \{p_0^1, \ldots, p_0^M\}$. Denote by $T(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}(p_0))$ the cost of discovering DCRs with the same consequence p_0, *i.e.,* $T(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}(p_0)) = O(c_{\mathsf{valid}} I_{\mathsf{max}} F |P_X|)$ (see Section V-C).

Since in practice, the number $|\mathcal{P}_0|$ work units is typically much larger than the number n of workers, we can evenly partition the workload by partitioning $\mathcal{P}_0$. By evenly balancing the workload, the parallel cost of PDCRLeaner is $O(\frac{T(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}(p_0^1)) + \ldots + T(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}(p_0^M))}{n}) = O(\frac{t(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req})}{n})$, where $t(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}) = T(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}(p_0^1)) + \ldots + T(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req}(p_0^M))$ since sequential DCRLeaner processes consequences in $\mathcal{P}_0$ one by one. The communication cost is bounded by $O(|\mathcal{D}| + |\mathcal{U}|)$ and thus, the computation cost, namely, $t(|\mathcal{D}|, |\mathcal{U}|, \mathsf{req})$, is dominating. Moreover, in practice, the costs for pre-computation, workload distribution, and result collection at the coordinator are also significantly lower than the discovery cost, which thus dominates the overall complexity.

This completes the proof of parallel scalability. □